



Universidad Autónoma de San Luis Potosí

Facultad de Ingeniería



Área de Ciencias de la Computación

Proyecto:

Servicios De Mantenimiento

Manual del Programador

Nombre de los alumnos:

- González Medina Claudia Karina
- Hernández López Eduardo Alejandro
- Piña Silva Juana María

Materia: Administración de Bases de Datos

Nombre del profesor: M.E.M. Claudia Alicia Méndez Hernández

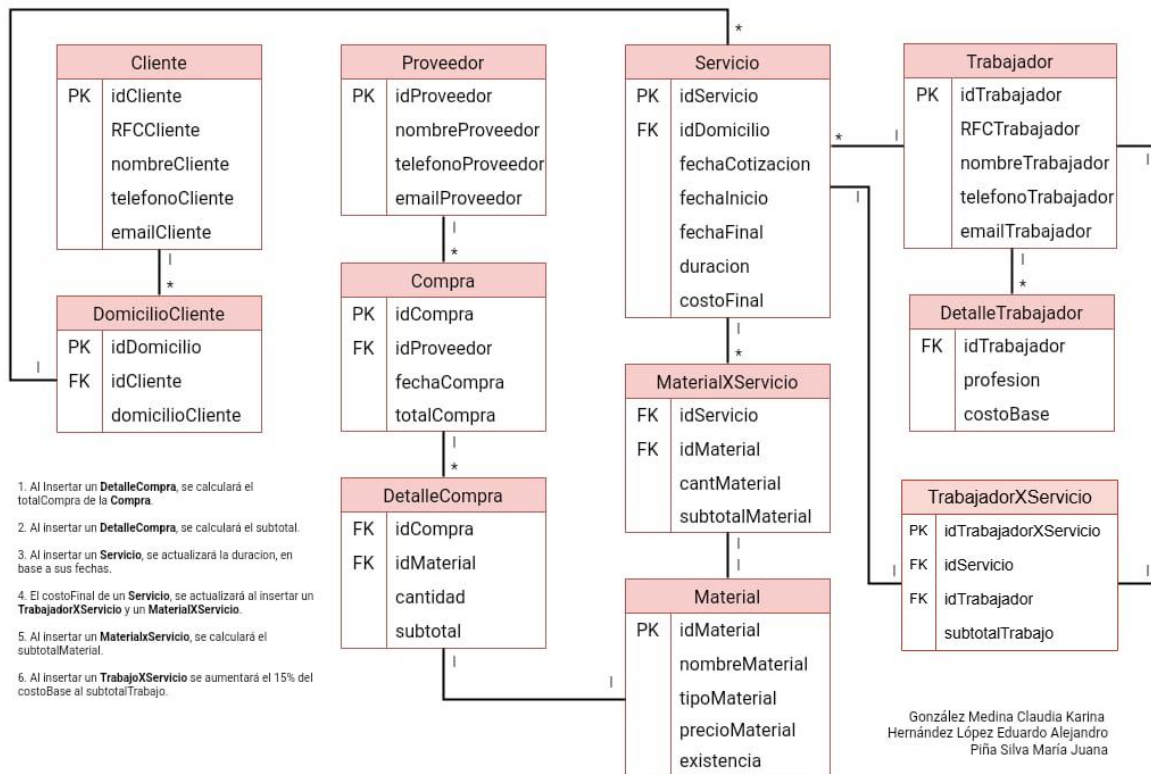
Número de Grupo: 231301

Nombre del Semestre: 2024-2025/I

Horario: 13:00 a 14:00 hrs

Para comenzar con el proyecto de Servicios de Mantenimiento necesitamos primeramente tener el Modelo Relación la cual es una herramienta para el modelo de datos donde muestra como interactúan las entidades y permiten a los desarrolladores a visualizar de mejor manera la relación entre los elementos claves y su cardinalidad.

Servicios de Mantenimiento



Ahora que sabemos cómo será nuestro proyecto es necesario implementar un script en MS SQL Server el cual es un sistema de administración de bases de datos relacional que contendrá la base de datos con la cual vamos a trabajar y posteriormente conectar con Visual Studio.

```
USE ServiciosdeMantenimiento
```

```
--Esquemas con los que se trabajó el proyecto
```

```
--para gestionar clientes, proveedores, compras y domicilios.
```

```
CREATE SCHEMA Comercial
```

```
--para gestionar servicios, trabajadores, materiales y las relaciones entre ellos.
```

```
CREATE SCHEMA Operativo
```

```
--TABLAS PARA EL PROYECTO
```

```
--CLIENTE
```

```
CREATE TABLE Comercial.Cliente
```

```
(
    idCliente BIGINT IDENTITY(1,1) NOT NULL,
    RFCCliente VARCHAR(20) NOT NULL,
    nombreCliente VARCHAR(100) NOT NULL,
    telefonoCliente VARCHAR(10) NOT NULL,
    emailCliente VARCHAR(200) NOT NULL,
    CONSTRAINT PK_CLIENTE PRIMARY KEY
(idCliente)
)
```

```
--DOMICILIOCLIENTE
```

```
CREATE TABLE Comercial.DomicilioCliente
```

```
(
    idDomicilio BIGINT IDENTITY(1,1) NOT NULL,
    idCliente BIGINT NOT NULL,
    domicilioCliente VARCHAR(200) NOT NULL,
    CONSTRAINT PK_DOMICILIO PRIMARY KEY
(idDomicilio),
    CONSTRAINT FK_CLIENTE01 FOREIGN
KEY(idCliente)
REFERENCES Comercial.Cliente(idCliente)
)
```

```
--PROVEEDOR
```

```
CREATE TABLE Comercial.Proveedor
```

```
(
    idProveedor BIGINT IDENTITY(1,1) NOT NULL,
    nombreProveedor VARCHAR(200) NOT NULL,
    telefonoProveedor VARCHAR(10) NOT NULL,
    emailProveedor VARCHAR(200) NOT NULL,
    CONSTRAINT PK_PROVEEDOR PRIMARY KEY
(idProveedor)
)
```

```
ALTER TABLE Comercial.Proveedor ADD CONSTRAINT
UQ_TELEFONOPROVEEDORUNICO UNIQUE
(telefonoProveedor);
```

```
ALTER TABLE Comercial.Proveedor ADD CONSTRAINT
UQ_EMAILPROVEEDORUNICO UNIQUE
(emailProveedor);
```

```
-- COMPRA
```

```
CREATE TABLE Comercial.Compra
```

```
(
    idCompra BIGINT IDENTITY(1,1) NOT NULL,
    idProveedor BIGINT NOT NULL,
    fechaCompra DATE NOT NULL,
    totalCompra MONEY,
    CONSTRAINT PK_COMPRA PRIMARY KEY
(idCompra),
    CONSTRAINT FK_PROVEEDOR01 FOREIGN
KEY(idProveedor)
REFERENCES
Comercial.Proveedor(idProveedor)
)
```

```
CREATE TABLE Comercial.DetalleCompra
```

```
(
    idCompra BIGINT NOT NULL,
    idMaterial BIGINT NOT NULL,
    cantidad INT NOT NULL,
    subtotal MONEY,
    CONSTRAINT FK_COMPRA FOREIGN
KEY(idCompra)
    REFERENCES Comercial.Compra(idCompra),
    CONSTRAINT FK_MATERIAL FOREIGN
KEY(idMaterial)
    REFERENCES Operativo.Material(idMaterial)
)
```

```
--SERVICIO
```

```
CREATE TABLE Operativo.Servicio
```

```
(
    idServicio BIGINT IDENTITY(1,1) NOT NULL,
    idDomicilio BIGINT NOT NULL,
    fechaCotizacion DATE NOT NULL,
    fechaInicio DATE NOT NULL,
    fechaFinal DATE NOT NULL,
    duracion INT,
    costoFinal DECIMAL,
    CONSTRAINT PK_SERVICIO PRIMARY KEY
(idServicio),
    CONSTRAINT FK_DOMICILIO FOREIGN KEY
(idDomicilio)
    REFERENCES Comercial.DomicilioCliente
(idDomicilio)
)
```

```
--MATERIAL
```

```
CREATE TABLE Operativo.Material
```

```
(
    idMaterial BIGINT IDENTITY(1,1) NOT NULL,
    nombreMaterial VARCHAR(100) NOT NULL,
    tipoMaterial VARCHAR(50) NOT NULL,
    precioMaterial MONEY NOT NULL,
    existencia INT NOT NULL,
    CONSTRAINT FK_MATERIAL PRIMARY KEY
(idMaterial)
)
```

```
ALTER TABLE Operativo.Material ADD CONSTRAINT
UQ_NOMBREMATERIALUNICO UNIQUE
(nombreMaterial);
```

```
CREATE TABLE Operativo.MaterialXServicio
```

```
(
    idServicio BIGINT NOT NULL,
    idMaterial BIGINT NOT NULL,
    cantMaterial INT NOT NULL,
    subtotalMaterial DECIMAL,
    CONSTRAINT FK_SERVICIO FOREIGN KEY
(idServicio)
    REFERENCES Operativo.Servicio (idServicio),
    CONSTRAINT FK_MATERIAL2 FOREIGN KEY
(idMaterial)
    REFERENCES Operativo.Material (idMaterial)
)
```

--TRABAJADOR

CREATE TABLE Operativo.Trabajador

```
(
    idTrabajador BIGINT IDENTITY (1,1) NOT NULL,
    RFCTrabajador VARCHAR(13) NOT NULL,
    nombreTrabajador VARCHAR(100) NOT NULL,
    telefonoTrabajador VARCHAR(10) NOT NULL,
    emailTrabajador VARCHAR(200) NOT NULL
    CONSTRAINT PK_TRABAJADOR PRIMARY KEY
(idTrabajador)
)

ALTER TABLE Operativo.Trabajador ADD CONSTRAINT
UQ_RFCTRAAJADORUNICO UNIQUE (RFCTrabajador);

ALTER TABLE Operativo.Trabajador ADD CONSTRAINT
UQ_TELEFONOTRAAJADORUNICO UNIQUE
(telefonoTrabajador);

ALTER TABLE Operativo.Trabajador ADD CONSTRAINT
UQ_EMAILTRAAJADORUNICO UNIQUE (emailTrabajador);
```

CREATE TABLE Operativo.DetalleTrabajador

```
(
    idTrabajador BIGINT NOT NULL,
    profesion VARCHAR(100) NOT NULL,
    costoBase MONEY NOT NULL

    CONSTRAINT FK_TRABAJADOR FOREIGN KEY
(idTrabajador)

REFERENCES Operativo.Trabajador(idTrabajador)
)
```

CREATE TABLE Operativo.TrabajadorXServicio

```
(
    idTrabajadorXServicio BIGINT IDENTITY (1,1)
NOT NULL,
    idServicio BIGINT NOT NULL,
    idTrabajador BIGINT NOT NULL,
    subtotalTrabajo MONEY

    CONSTRAINT PK_TRABAJADORXSERVICIO
PRIMARY KEY (idTrabajadorXServicio),

    CONSTRAINT FK_SERVICIO2 FOREIGN KEY
(idServicio)

REFERENCES Operativo.Servicio(idServicio),

    CONSTRAINT FK_TRABAJADOR2 FOREIGN
KEY (idTrabajador)

REFERENCES
Operativo.Trabajador(idTrabajador)
)
```

--REGLAS

```
CREATE RULE RL_TIPO_MATERIAL AS @TIPO IN
('Plomeria', 'Reconstruccion', 'Electricidad')

EXEC sp_bindrule 'RL_TIPO_MATERIAL',
'Operativo.Material.tipoMaterial'

CREATE RULE RL_CANT_MATERIAL AS @CANTIDAD >=1

EXEC sp_bindrule
'RL_CANT_MATERIAL','Comercial.DetalleCompra.cantid
ad'
```

```

--Disparadores para el proyecto

--DISPARADOR(ES) PARA DETALLECOMPRA

CREATE TRIGGER
TGR_CALCULAR_SUBTOTAL_DETALLECOMPRA

ON Comercial.DetalleCompra

AFTER INSERT, UPDATE

AS

BEGIN

    DECLARE @idCompra BIGINT;

    DECLARE @idMaterial BIGINT;

    DECLARE @cantidad INT;

    DECLARE @precioMaterial MONEY;

    DECLARE @subtotal DECIMAL(18, 2);

    -- Obtener los valores insertados o actualizados

    SELECT @idCompra = idCompra, @idMaterial =
idMaterial, @cantidad = cantidad

    FROM inserted;

    -- Obtener el precio del material

    SELECT @precioMaterial = precioMaterial

    FROM Operativo.Material

    WHERE idMaterial = @idMaterial;

    -- Calcular el subtotal

    SET @subtotal = @precioMaterial * @cantidad;

    -- Actualizar el campo subtotal en la tabla DetalleCompra

    UPDATE Comercial.DetalleCompra

    SET subtotal = @subtotal

    WHERE idCompra = @idCompra AND idMaterial =
@idMaterial;

END;

CREATE TRIGGER TGR_ACTUALIZATOTALCOMPRA

ON Comercial.DetalleCompra

AFTER INSERT, UPDATE, DELETE

AS

```

```

BEGIN

    -- Declarar variables

    DECLARE @idCompra INT;

    -- Calcular el nuevo total de la compra sumando
    todos los subtotales de los DetalleCompra asociados

    DECLARE @nuevoTotal DECIMAL(18, 2);

    -- Verificar si es una inserción o actualización

    IF EXISTS (SELECT 1 FROM inserted)

    BEGIN

        -- Obtener el idCompra de los registros insertados o
        actualizados

        SELECT @idCompra = (SELECT TOP 1 idCompra
        FROM inserted);

        -- Calcular el nuevo total

        SELECT @nuevoTotal = ISNULL(SUM(subtotal), 0)

        FROM Comercial.DetalleCompra

        WHERE idCompra = @idCompra;

        -- Actualizar el totalCompra en la tabla
        Comercial.Compra

        UPDATE Comercial.Compra

        SET totalCompra = @nuevoTotal

        WHERE idCompra = @idCompra;

    END

    -- Verificar si es una eliminación

    IF EXISTS (SELECT 1 FROM deleted)

    BEGIN

        -- Obtener el idCompra de los registros eliminados

        SELECT @idCompra = (SELECT TOP 1 idCompra
        FROM deleted);

        SELECT @nuevoTotal = ISNULL(SUM(subtotal), 0)

        FROM Comercial.DetalleCompra

        WHERE idCompra = @idCompra;

```

```

-- Actualizar el totalCompra en la tabla
Comercial.Compra

UPDATE Comercial.Compra

SET totalCompra = @nuevoTotal

WHERE idCompra = @idCompra;

END

END;

CREATE TRIGGER TGR_ACTUALIZA_MATERIAL

ON Comercial.DetalleCompra

AFTER INSERT, UPDATE, DELETE

AS

BEGIN

-- Manejo de inserciones

IF EXISTS (SELECT * FROM inserted) AND NOT EXISTS
(SELECT * FROM deleted)

BEGIN

-- Actualizar la existencia sumando las nuevas
cantidades

UPDATE Operativo.Material

SET existencia = existencia + i.cantidad

FROM inserted i

WHERE Operativo.Material.idMaterial = i.idMaterial;

END

-- Manejo de actualizaciones

IF EXISTS (SELECT * FROM deleted) AND EXISTS (SELECT *
FROM deleted)

BEGIN

-- Actualizar la existencia restando las antiguas
cantidades

UPDATE Operativo.Material

SET existencia = existencia - d.cantidad + i.cantidad

FROM deleted d

JOIN inserted i ON d.idMaterial = i.idMaterial

```

```

WHERE Operativo.Material.idMaterial =
d.idMaterial;

END

-- Manejo de eliminaciones

IF EXISTS (SELECT * FROM deleted) AND NOT EXISTS
(SELECT * FROM inserted)

BEGIN

-- Restar las cantidades al eliminar registros de
DetalleCompra

UPDATE Operativo.Material

SET existencia = existencia - d.cantidad

FROM deleted d

WHERE Operativo.Material.idMaterial =
d.idMaterial;

END

END;

--DISPARADOR(ES) PARA SERVICIO

CREATE TRIGGER TGR_UPDATE_COSTO_FINAL

ON Operativo.Servicio

AFTER INSERT, UPDATE

AS

BEGIN

-- Evita la recursión verificando el nivel de anidación

IF (TRIGGER_NESTLEVEL() > 1) RETURN;

DECLARE @idServicio BIGINT;

DECLARE @subtotalMaterial DECIMAL(18, 2) = 0;

DECLARE @subtotalTrabajo DECIMAL(18, 2) = 0;

DECLARE @nuevoCostoFinal DECIMAL(18, 2);

DECLARE @costoActual DECIMAL(18, 2);

-- Obtener el idServicio del registro afectado

SELECT @idServicio = idServicio FROM inserted;

```

```

-- Calcular los subtotales

SELECT @subtotalMaterial =
ISNULL(SUM(subtotalMaterial), 0)

FROM Operativo.MaterialXServicio

WHERE idServicio = @idServicio;

SELECT @subtotalTrabajo =
ISNULL(SUM(subtotalTrabajo), 0)

FROM Operativo.TrabajadorXServicio

WHERE idServicio = @idServicio;

-- Calcular el nuevo costo final

SET @nuevoCostoFinal = @subtotalMaterial +
@subtotalTrabajo;

-- Obtener el costo actual

SELECT @costoActual = costoFinal

FROM Operativo.Servicio

WHERE idServicio = @idServicio;

-- Actualizar solo si el costo ha cambiado

IF (@nuevoCostoFinal <> @costoActual)

BEGIN

    UPDATE Operativo.Servicio

    SET costoFinal = @nuevoCostoFinal

    WHERE idServicio = @idServicio;

END

END;

CREATE TRIGGER TGR_DURACIONSERVICIO

ON Operativo.Servicio

AFTER INSERT, UPDATE

AS

BEGIN

    -- Evita la recursión verificando el nivel de
    anidación

    IF (TRIGGER_NESTLEVEL() > 1) RETURN;

```

```

-- Actualizar la duración en base a las fechas

UPDATE Operativo.Servicio

SET duracion = DATEDIFF(DAY, i.fechaInicio,
i.fechaFinal)

FROM Operativo.Servicio S

INNER JOIN inserted I ON S.idServicio = I.idServicio;

END;

--DISPARADOR(ES) PARA MATERIAL

CREATE TRIGGER TGR_CALCULA_SUBTOTAL_MATERIAL

ON Operativo.MaterialXServicio

AFTER INSERT, UPDATE, DELETE

AS

BEGIN

    -- Manejo de inserciones (solo INSERT)

    IF EXISTS (SELECT * FROM inserted) AND NOT EXISTS
(SELECT * FROM deleted)

    BEGIN

        -- Actualiza el subtotal multiplicando la cantidad
        por el costo del material

        UPDATE Operativo.MaterialXServicio

        SET subtotalMaterial = i.cantMaterial *
m.precioMaterial

        FROM inserted i

        JOIN Operativo.Material m ON i.idMaterial =
m.idMaterial

        WHERE Operativo.MaterialXServicio.idMaterial =
i.idMaterial;

    END

```



```

--DISPARADOR(ES) PARA MATERIAL

CREATE TRIGGER
TGR_CALCULA_SUBTOTAL_MATERIAL

ON Operativo.MaterialXServicio

AFTER INSERT, UPDATE, DELETE

AS

BEGIN

    -- Manejo de inserciones (solo INSERT)

    IF EXISTS (SELECT * FROM inserted) AND NOT
    EXISTS (SELECT * FROM deleted)

        BEGIN

            -- Actualiza el subtotal multiplicando la cantidad
            por el costo del material

            UPDATE Operativo.MaterialXServicio

            SET subtotalMaterial = i.cantMaterial *
            m.precioMaterial

            FROM inserted i

            JOIN Operativo.Material m ON i.idMaterial =
            m.idMaterial

            WHERE Operativo.MaterialXServicio.idMaterial =
            i.idMaterial;

        END

    -- Manejo de actualizaciones (UPDATE)

    IF EXISTS (SELECT * FROM inserted) AND EXISTS
    (SELECT * FROM deleted)

        BEGIN

            -- Actualiza el subtotal, ajustando la cantidad
            antigua y nueva

            UPDATE Operativo.MaterialXServicio

            SET subtotalMaterial = (i.cantMaterial *
            m.precioMaterial)

            FROM inserted i

            JOIN deleted d ON d.idMaterial = i.idMaterial

            JOIN Operativo.Material m ON i.idMaterial =
            m.idMaterial

```

```

WHERE Operativo.MaterialXServicio.idMaterial =
i.idMaterial;

END

-- Manejo de eliminaciones (DELETE)

IF EXISTS (SELECT * FROM deleted) AND NOT EXISTS
(SELECT * FROM inserted)

BEGIN

    -- Si se elimina un material, no es necesario hacer
    nada con el subtotal en la tabla MaterialXServicio, ya
    que no será modificado

    -- Aquí podrías agregar lógica si deseas realizar
    algún tipo de ajuste al subtotal al eliminar,
    dependiendo de tu lógica de negocio.

    DELETE FROM Operativo.MaterialXServicio

    WHERE idMaterial IN (SELECT idMaterial FROM
    deleted);

END

END;

CREATE TRIGGER
TRG_UPDATECOSTOFINAL_MATERIALXSERVICIO

ON Operativo.MaterialXServicio

AFTER INSERT, UPDATE, DELETE

AS

BEGIN

    DECLARE @idServicio BIGINT;

    -- Obtener el idServicio del registro afectado

    SELECT @idServicio = COALESCE(i.idServicio,
    d.idServicio)

    FROM inserted i

    FULL OUTER JOIN deleted d ON i.idServicio =
    d.idServicio;

```

-- Calcular el costo final sumando los subtotales
de MaterialXServicio y TrabajadorXServicio

UPDATE Operativo.Servicio

SET costoFinal =

(SELECT COALESCE(SUM(subtotalMaterial), 0)

FROM Operativo.MaterialXServicio

WHERE idServicio = @idServicio)

+

(SELECT COALESCE(SUM(subtotalTrabajo), 0)

FROM Operativo.TrabajadorXServicio

WHERE idServicio = @idServicio)

WHERE idServicio = @idServicio;

END;

CREATE TRIGGER

TGR_ACTUALIZAR_EXISTENCIAS_MATERIAL

ON Operativo.MaterialXServicio

AFTER INSERT, UPDATE, DELETE

AS

BEGIN

-- Manejo de inserciones

IF EXISTS (SELECT * FROM inserted) AND NOT
EXISTS (SELECT * FROM deleted)

BEGIN

-- Actualizar la existencia sumando las nuevas
cantidades

UPDATE Operativo.Material

SET existencia = existencia + i.cantMaterial

FROM inserted i

WHERE Operativo.Material.idMaterial =
i.idMaterial;

END

-- Manejo de eliminaciones

IF EXISTS (SELECT * FROM deleted) AND NOT EXISTS
(SELECT * FROM inserted)

BEGIN

-- Restar las cantidades al eliminar registros de
DetalleCompra

UPDATE Operativo.Material

SET existencia = existencia - d.cantMaterial

FROM deleted d

WHERE Operativo.Material.idMaterial =
d.idMaterial;

END

END;

--DISPARADOR(ES) PARA TRABAJADOR

CREATE TRIGGER TGR_CALCULASUBTOTALTRABAJO

ON Operativo.TrabajadorXServicio

AFTER INSERT, UPDATE

AS

BEGIN

SET NOCOUNT ON;

UPDATE txs

SET txs.subtotalTrabajo = ISNULL(dt.costoBase *
1.15, 0)

FROM Operativo.TrabajadorXServicio txs

INNER JOIN Operativo.DetalleTrabajador dt ON
txs.idTrabajador = dt.idTrabajador

INNER JOIN inserted i ON i.idServicio = txs.idServicio
AND i.idTrabajador = txs.idTrabajador;

END;

CREATE TRIGGER	-- Calcular el costo final sumando los subtotales de
TRG_UPDATECOSTOFINAL_TRABAJADORXSERVICIO	MaterialXServicio y TrabajadorXServicio
ON Operativo.TrabajadorXServicio	UPDATE Operativo.Servicio
AFTER INSERT, UPDATE, DELETE	SET costoFinal =
AS	(SELECT COALESCE(SUM(subtotalMaterial), 0)
BEGIN	FROM Operativo.MaterialXServicio
DECLARE @idServicio BIGINT;	WHERE idServicio = @idServicio)
-- Obtener el idServicio del registro insertado,	+
actualizado o eliminado	(SELECT COALESCE(SUM(subtotalTrabajo), 0)
SELECT @idServicio = COALESCE((SELECT	FROM Operativo.TrabajadorXServicio
idServicio FROM inserted), (SELECT idServicio FROM	WHERE idServicio = @idServicio)
deleted));	WHERE idServicio = @idServicio;
	END;

El anterior script contiene los principales elementos:

- A. La creación de la Base de Datos (DATABASE ServiciosDeMantenimiento)
- B. Creación de dos esquemas (SCHEMA Comercial, SCHEMA Operativo)
- C. Creación de las tablas (TABLE) con sus Primary Key (PK) y Foreign Key (FK).
En algunas de ellas cuentan con restricciones de tipo UNIQUE las cuales evitan duplicación de información y de tipo NOT NULL donde evita valores nulos.
 - 1. Cliente
 - 2. DomicilioCliente
 - 3. Compra
 - 4. DetalleCompra
 - 5. DetalleTrabajador
 - 6. Material
 - 7. MaterialXServicio
 - 8. Proveedor
 - 9. Trabajador
 - 10. TrabajadorXServicio
 - 11. Servicio
- D. Creación de dos reglas (RULE) vinculadas con columnas de alguna tabla (EXEC sp_bindrule)
 - 1. RULE RL_TIPO_MATERIAL (Tipo cadena)
 - 2. RULE RL_CANT_MATERIAL (Numérica)

E. Creación de los disparadores los cuales buscan automatizar procesos y se ejecutan de manera automática cuando se produce un evento relacionado con las tablas con las cuales está asociado.

1. TGR_CALCULAR_SUBTOTAL_DETALLECOMPRA
2. TGR_ACTUALIZATOTALCOMPRA
3. TGR_ACTUALIZA_MATERIAL
4. TGR_UPDATE_COSTO_FINAL
5. TRIGGER TGR_DURACIONSERVICIO
6. TGR_CALCULA_SUBTOTAL_MATERIAL
7. TRG_UPDATECOSTOFINAL_MATERIALXSERVICIO
8. TGR_ACTUALIZAR_EXISTENCIAS_MATERIAL
9. TGR_CALCULASUBTOTALTRABAJO
10. TRG_UPDATECOSTOFINAL_TRABAJADORXSERVICIO

Y llega el momento de realizar la conexión con Visual Studio. Para lograr dicha conexión será necesario crear un nuevo proyecto de la plantilla: Aplicación de Windows Forms (.NET Framework).

En cada formulario de este proyecto contendrá cuatro aspectos principales: la consulta de los datos, la inserción de un nuevo dato con alguna restricción para evitar incoherencia e inconsistencia entre los datos, la modificación de un dato que de igual manera como sucede con la inserción se maneja con alguna restricción para evitar incoherencia e inconsistencia entre los datos y la eliminación de un dato donde en algunos casos su eliminación se realiza en forma de cascada para evitar errores.

De manera más explícita les explicaremos en que consiste cada uno de los aspectos anteriormente mencionados.

En la consulta de los datos, se define la cadena de consulta SQL almacenada en una variable local, verificación y creación de las columnas de la tabla donde si todavía no existen columnas, se crean manualmente y el cumplimentar de la tabla donde se añaden los datos y se asignan a las celdas correspondientes.

En la inserción de un dato, la asignación de los valores que fueron ingresados por medio de las herramientas del cuadro de herramientas, definición de la cadena de inserción SQL almacenada en una variable local en la cual los valores se asignan a las variables por medio de parámetros, donde en algunos casos no permite su inserción por restricciones de tipo UNIQUE las cuales fueron declaradas en el script para evitar duplicidad o en algunos casos la coherencia en las fechas donde no permite su inserción cuando su fecha de finalización es menor que su fecha de inicialización.

En la modificación de un dato, al seleccionar una tupla utilizando el evento CellClick del control DataGridView el cual se produce cuando se da clic en cualquier parte de una celda y asigna los valores a las herramientas del cuadro de herramientas,

definición de la cadena de actualización almacenada en una variable local por medio de parámetros, y comprende los mismos tipos de restricciones que en la inserción de un dato.

Y por último, pero no menos importante, en la eliminación de un dato, al seleccionar una tupla igualmente como sucede en la modificación asigna los valores a las herramientas del cuadro de herramientas, definición de la cadena de eliminación almacenada en una variable local donde en ciertos casos elimina el registro en forma de cascada cuando dicha tabla está relacionada con una tabla de detalle y reinicia su ID cuando la tabla no contiene tuplas.

Código documentado para la conexión.

```
// Librerías del sistema y .NET Framework usadas para crear y gestionar la conexión con SQL
```

```
using System;
```

```
using System.Data.SqlClient;
```

```
using System.Windows.Forms;
```

```
/*
```

```
    Clase para gestionar la conexión con la base de datos SQL Server.
```

```
    Proporciona métodos para abrir y cerrar la conexión.
```

```
*/
```

```
public class ConexionDB
```

```
{
```

```
    private SqlConnection conexion;
```

```
    private readonly string cadenaConexion;
```

```
/*
```

```
    Constructor de la clase ConexionDB
```

```
    Inicializa la conexión con la base de datos a través de una cadena proporcionada.
```

```
*/
```

```
public ConexionDB(string connectionString)
```

```
{
```

```
    cadenaConexion = connectionString;
```

```
    conexion = new SqlConnection(cadenaConexion);
```

```
}
```

```
/*
```

```
    Abre la conexión con la base de datos si está cerrada.
```

Muestra un mensaje de error si ocurre una excepción al abrir la conexión.

*/

```
public SqlConnection AbrirConexion()
```

```
{
```

```
    try
```

```
    {
```

```
        if (conexion.State == System.Data.ConnectionState.Closed)
```

```
        {
```

```
            conexion.Open();
```

```
        }
```

```
    }
```

```
    catch (SqlException ex)
```

```
    {
```

```
        MessageBox.Show("Error al abrir la conexión: " + ex.Message);
```

```
    }
```

```
    return conexion;
```

```
}
```

/*

Cierra la conexión con la base de datos si está abierta.

Muestra un mensaje de error si ocurre una excepción al cerrar la conexión.

*/

```
public void CerrarConexion()
```

```
{
```

```
    try
```

```
    {
```

```
        if (conexion.State == System.Data.ConnectionState.Open)
```

```
        {
```

```
            conexion.Close();
```

```
        }
```

```
    }
```

```
    catch (SqlException ex)
```

```
    {
```

```
        MessageBox.Show("Error al cerrar la conexión: " + ex.Message);
```

```
    }
```

```
}
```

```
}
```

Formulario ServiciosDeMantenimiento (Menú).



Código documentado del Formulario ServiciosDeMantenimiento (Menú).

// Librerías del sistema y .NET Framework usadas para crear y gestionar el formulario Principal

```
using System;
```

```
using System.Collections.Generic;
```

```
using System.ComponentModel;
```

```
using System.Data;
```

```
using System.Drawing;
```

```
using System.Linq;
```

```
using System.Text;
```

```
using System.Threading.Tasks;
```

```
using System.Windows.Forms;
```

```
namespace ServiciosDeMantenimiento
```

```
{
```

```
    /*
```

```
        Clase principal del formulario de Servicios de Mantenimiento.
```

```
        Este formulario actúa como el menú principal de la aplicación, proporcionando acceso a otros formularios específicos.
```

```
        Los siguiente formularios son:
```

```
        - Formulario de Material.
```

```
        - Formulario de Cliente.
```

```
        - Formulario de Proveedor.
```

```
        - Formulario de Trabajador.
```

```
        - Formulario de Compra.
```

```
        - Formulario de Servicio.
```

```
    */
```

```
    public partial class ServiciosdeMantenimiento : Form
```

```
{
```

```
    /*
```

Constructor del formulario principal.

Inicializa los componentes del formulario.

*/

```
public ServiciosdeMantenimiento()
```

```
{
```

```
    InitializeComponent();
```

```
}
```

/*

Evento que se dispara al hacer clic en el botón de Material.

Abre el formulario de Material como un cuadro de diálogo, permitiendo gestionar los datos de materiales.

*/

```
private void btnMaterial_Click(object sender, EventArgs e)
```

```
{
```

```
    Material materialForm = new Material();
```

```
    materialForm.ShowDialog();
```

```
}
```

/*

Evento que se dispara al hacer clic en el botón de Cliente.

Abre el formulario de Cliente como un cuadro de diálogo, permitiendo gestionar los datos de los clientes.

*/

```
private void btnCliente_Click(object sender, EventArgs e)
```

```
{
```

```
    Cliente clienteForm = new Cliente();
```

```
    clienteForm.ShowDialog();
```

```
}
```

/*

Evento que se dispara al hacer clic en el botón de Proveedor.

Abre el formulario de Proveedor como un cuadro de diálogo, permitiendo gestionar los datos de los proveedores.

*/

```
private void btnProveedor_Click(object sender, EventArgs e)
```

```
{
```

```
    Proveedor proveedorForm = new Proveedor();
```

```
    proveedorForm.ShowDialog();
```

```
}
```



```

/*
    Evento que se dispara al hacer clic en el botón de Trabajador.

    Abre el formulario de Trabajador como un cuadro de diálogo, permitiendo gestionar los datos de los trabajadores.
*/
private void btnTrabajador_Click(object sender, EventArgs e)
{
    Trabajador trabajadorForm = new Trabajador();
    trabajadorForm.ShowDialog();
}

/*
    Evento que se dispara al hacer clic en el botón de Compra.

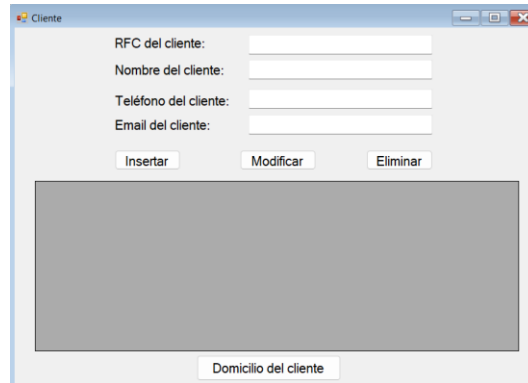
    Abre el formulario de Compra como un cuadro de diálogo, permitiendo gestionar los datos de las compras.
*/
private void btnCompra_Click(object sender, EventArgs e)
{
    Compra compraForm = new Compra();
    compraForm.ShowDialog();
}

/*
    Evento que se dispara al hacer clic en el botón de Servicio.

    Abre el formulario de Servicio como un cuadro de diálogo, permitiendo gestionar los datos de los servicios.
*/
private void btnServicio_Click(object sender, EventArgs e)
{
    Servicio servicioForm = new Servicio();
    servicioForm.ShowDialog();
}
}
}

```

Formulario Cliente.



Código documentado del Formulario Cliente.

// Librerías del sistema y .NET Framework usadas para crear y gestionar el formulario Cliente.

```
using System;

using System.Collections.Generic;

using System.ComponentModel;

using System.Data;

using System.Data.SqlClient;

using System.Drawing;

using System.Linq;

using System.Text;

using System.Text.RegularExpressions;

using System.Threading.Tasks;

using System.Windows.Forms;

namespace ServiciosDeMantenimiento

{

    /*

        Clase para gestionar el manejo de los cliente dentro de los Servicios de Mantenimiento

        Proporciona métodos para consultar, insertar, modificar y eliminar datos.

    */

    public partial class Cliente : Form

    {

        // Crea una instancia de una nueva conexión para conectarse con la base de datos de SQL Server.

        ConexionDB nuevaConexion = new ConexionDB("Server=ASUS\\SQLEXPRESS1; DATABASE = ServiciosdeMantenimiento; " +

            "integrated security = true");

        int idCliente = -1;
```

```

string rfcCliente = " ";
string nomCliente = " ";
string telCliente = " ";
string emailCliente = " ";

//Lista que almacena objetos de tipo DomicilioCliente.
private List<DomicilioCliente> domicilioClienteForms = new List<DomicilioCliente>();

/*
    Constructor del formulario Cliente.
    Inicializa el formulario y carga los datos de la tabla Cliente en el DataGridView.
    También suscribe el evento CellClick de la tabla.
*/
public Cliente()
{
    InitializeComponent();
    ConsultaDatos();
    TablaCliente.CellClick += TablaCliente_CellClick;
}

/*
    ConsultaDatos
    Recupera y muestra todos los registros de la tabla Comercial.Cliente en el DataGridView.
    Formatea las columnas y limpia las filas antes de agregar los datos.
*/
public void ConsultaDatos()
{
    nuevaConexion.AbrirConexion();
    TablaCliente.DataSource = null;

    // Cadena SQL para consultar la tabla Comercial.Cliente.
    String selectInfo = "SELECT * FROM Comercial.Cliente";
    SqlCommand scm = new SqlCommand(selectInfo, nuevaConexion.AbrirConexion());
    SqlDataReader lector = scm.ExecuteReader();

    // Agrega columnas en la TablaCliente si es que no contiene columnas.
    if (TablaCliente.Columns.Count == 0)

```

```

{
    TablaCliente.Columns.Add("idCliente", "ID");

    TablaCliente.Columns.Add("RFCCliente", "RFC del Cliente");

    TablaCliente.Columns.Add("nombreCliente", "Nombre del Cliente");

    TablaCliente.Columns.Add("telefonoCliente", "Teléfono del Cliente");

    TablaCliente.Columns.Add("emailCliente", "Email");
}

TablaCliente.Rows.Clear();

int i = 0;

// Agrega a TablaCliente una nueva fila para almacenar con los valores correspondientes a las columnas.
while (lector.Read())
{
    TablaCliente.Rows.Add();

    TablaCliente.Rows[i].Cells[0].Value = lector["idCliente"].ToString();

    TablaCliente.Rows[i].Cells[1].Value = lector["RFCCliente"].ToString();

    TablaCliente.Rows[i].Cells[2].Value = lector["nombreCliente"].ToString();

    TablaCliente.Rows[i].Cells[3].Value = lector["telefonoCliente"].ToString();

    TablaCliente.Rows[i].Cells[4].Value = lector["emailCliente"].ToString();

    i++;
}

lector.Close();

nuevaConexion.CerrarConexion();
}

/*
    TablaCliente_CellClick

    Evento que se activa al hacer clic en una celda del DataGridView.

    Llena los campos de entrada con los datos de la fila seleccionada.
*/
private void TablaCliente_CellClick(object sender, DataGridViewCellEventArgs e)
{
    if (e.RowIndex >= 0)
    {
        DataGridViewRow row = TablaCliente.Rows[e.RowIndex];

        idCliente = int.Parse(row.Cells[0].Value.ToString());
    }
}

```

```

        txtRFCCliente.Text = row.Cells[1].Value.ToString();

        txtNomCliente.Text = row.Cells[2].Value.ToString();

        txtTelCliente.Text = row.Cells[3].Value.ToString();

        txtEmailCliente.Text = row.Cells[4].Value.ToString();

        // Hace visible al botón btnDomicilio.

        btnDomicilio.Visible = true;

    }

}

/*
InsertaDato

Inserta un nuevo registro en la tabla Comercial.Cliente con los datos ingresados.

Si ocurre un error (como duplicación de nombre), muestra un mensaje de error.

*/

public void InsertaDato()
{
    rfcCliente = txtRFCCliente.Text;
    nomCliente = txtNomCliente.Text;
    telCliente = txtTelCliente.Text;
    emailCliente = txtEmailCliente.Text;

    nuevaConexion.AbrirConexion();

    // Cadena SQL para agregar un nuevo registro en la tabla Comercial.Cliente.
    string InsertInfo = "INSERT INTO Comercial.Cliente(RFCCliente, nombreCliente, telefonoCliente, emailCliente)" +
        "VALUES('" + rfcCliente + "', '" + nomCliente + "', '" + telCliente + "', '" + emailCliente + "')";

    SqlCommand cm = new SqlCommand(InsertInfo, nuevaConexion.AbrirConexion());

    cm.ExecuteNonQuery();

    nuevaConexion.CerrarConexion();

}

/*
ModificaDato

Modifica el registro seleccionado en la tabla Comercial.Cliente con los datos actualizados.

Si ocurre un error (como duplicación de nombre), muestra un mensaje de error.

*/

```

```

public void ModificaDato()
{
    string verificaEmail = txtEmailCliente.Text;

    rfcCliente = txtRFCCliente.Text;
    nomCliente = txtNomCliente.Text;
    telCliente = txtTelCliente.Text;
    emailCliente = txtEmailCliente.Text;

    nuevaConexion.CerrarConexion();

    // Cadena SQL para modificar un nuevo registro en la tabla Comercial.Cliente.
    string update = "UPDATE Comercial.Cliente SET RFCCliente = @RFC, nombreCliente = @Nombre, telefonoCliente = @Telefono, emailCliente = @Email " +
        "WHERE idCliente = @Id";
    SqlCommand cmd = new SqlCommand(update, nuevaConexion.AbrirConexion());
    cmd.Parameters.AddWithValue("@RFC", rfcCliente);
    cmd.Parameters.AddWithValue("@Nombre", nomCliente);
    cmd.Parameters.AddWithValue("@Telefono", telCliente);
    cmd.Parameters.AddWithValue("@Email", emailCliente);
    cmd.Parameters.AddWithValue("@Id", idCliente);
    cmd.ExecuteNonQuery();
}

/*
EliminaDato
Elimina el registro seleccionado en la tabla Comercial.Cliente con los datos actualizados.
*/
public void EliminarDato()
{
    if (idCliente != -1)
    {
        nuevaConexion.AbrirConexion();

        // Cadena SQL para eliminar un nuevo registro en la tabla Comercial.Cliente.
        string delete = "DELETE FROM Comercial.Cliente WHERE idCliente = @Id";
    }
}

```

```

SqlCommand cmd = new SqlCommand(delete, nuevaConexion.AbrirConexion());

cmd.Parameters.AddWithValue("@Id", idCliente);

cmd.ExecuteNonQuery();


string checkEmptyQuery = "SELECT COUNT(*) FROM Comercial.Cliente";

SqlCommand checkCmd = new SqlCommand(checkEmptyQuery, nuevaConexion.AbrirConexion());

int rowCount = (int)checkCmd.ExecuteScalar();


// Reinicia el valor del idCliente cuando la cantidad de filas es igual a 0.
if (rowCount == 0)
{
    string resetIdentityQuery = "DBCC CHECKIDENT ('Comercial.Cliente', RESEED, 0)";

    SqlCommand resetCmd = new SqlCommand(resetIdentityQuery, nuevaConexion.AbrirConexion());

    resetCmd.ExecuteNonQuery();
}

nuevaConexion.CerrarConexion();
}
}

/*
btnInserta_Click

Evento de clic del botón de inserción.

Llama a InsertaDato, actualiza el DataGridView llamando a ConsultaDatos y limpia los campos de entrada.
*/

private void btnInserta_Click(object sender, EventArgs e)
{
    InsertaDato();

    ConsultaDatos();

    txtRFCCliente.Clear();

    txtNomCliente.Clear();

    txtTelCliente.Clear();

    txtEmailCliente.Clear();
}

/*
btnModifica_Click

Evento de clic del botón de modificación.

```

Llama a ModificaDato, actualiza el DataGridView llamando a ConsultaDatos y limpia los campos de entrada.

*/

```
private void btnModifica_Click(object sender, EventArgs e)
```

```
{
```

```
    ModificaDato();
```

```
    ConsultaDatos();
```

```
    txtRFCCliente.Clear();
```

```
    txtNomCliente.Clear();
```

```
    txtTelCliente.Clear(); ;
```

```
    txtEmailCliente.Clear();
```

```
}
```

/*

btnElimina_Click

Evento de clic del botón de eliminación.

Llama a EliminaDato, actualiza el DataGridView llamando a ConsultaDatos y limpia los campos de entrada.

*/

```
private void btnElimina_Click(object sender, EventArgs e)
```

```
{
```

```
    EliminarDato();
```

```
    ConsultaDatos();
```

```
    txtRFCCliente.Clear();
```

```
    txtNomCliente.Clear();
```

```
    txtTelCliente.Clear();
```

```
    txtEmailCliente.Clear();
```

```
}
```

/*

Evento que se dispara al hacer clic en el botón de Domicilio.

Abre el formulario DomicilioCliente para gestionar los domicilios asociados a un cliente específico.

*/

```
private void btnDomicilio_Click(object sender, EventArgs e)
```

```
{
```

```
    if (idCliente != -1)
```

```
    {
```

```
        DomicilioCliente domicilioClienteForm = new DomicilioCliente(idCliente);
```

```
        domicilioClienteForms.Add(domicilioClienteForm);
```

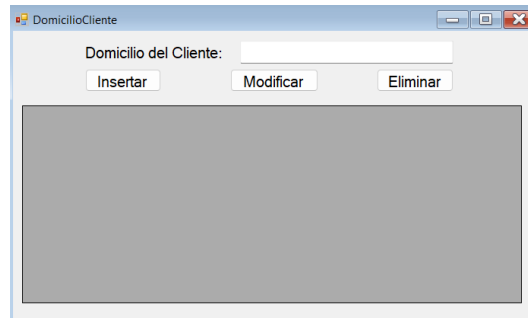


```

        domicilioClienteForm.Show();
    }
}
}
}

```

Formulario DomicilioCliente.



Código documentado del Formulario DomicilioCliente.

// Librerías del sistema y .NET Framework usadas para crear y gestionar el formulario DomicilioCliente

```

using System;

using System.Collections.Generic;

using System.ComponentModel;

using System.Data;

using System.Data.SqlClient;

using System.Drawing;

using System.Linq;

using System.Text;

using System.Threading.Tasks;

using System.Windows.Forms;

using System.Text.RegularExpressions;

```

```

namespace ServiciosDeMantenimiento

```

```

{

```

```

    /*

```

```

        Clase para gestionar el manejo de los domicilios de los cliente dentro de los Servicios de Mantenimiento

```

```

        Proporciona métodos para consultar, insertar, modificar y eliminar datos.

```

```

    */

```

```

    public partial class DomicilioCliente : Form

```

```

    {

```

```

        // Crea una instancia de una nueva conexión para conectarse con la base de datos de SQL Server.

```

```

ConexionDB nuevaConexion = new ConexionDB("Server=ASUS\\SQLEXPRESS1; DATABASE = ServiciosdeMantenimiento;" +
    "integrated security = true");

int idDomicilio = -1;

int idCliente;

string domicilioCliente = "";

/*
    Constructor del formulario DomicilioCliente.

    Inicializa el formulario y carga los datos de la tabla DomicilioCliente en el DataGridView.

    También suscribe el evento CellClick de la tabla.
*/

public DomicilioCliente(int idCliente)
{

    InitializeComponent();

    this.idCliente = idCliente;

    ConsultaDatos();

    TablaDomicilioCliente.CellClick += TablaDomicilioCliente_CellClick;

}

/*
    ConsultaDatos

    Recupera y muestra todos los registros de la tabla Comercial.DomicilioCliente en el DataGridView.

    Formatea las columnas y limpia las filas antes de agregar los datos.
*/

public void ConsultaDatos()
{

    nuevaConexion.AbrirConexion();

    TablaDomicilioCliente.DataSource = null;

    // Cadena SQL para consultar la tabla Comercial.DomicilioCliente.

    string selectInfo = "SELECT d.idDomicilio, d.idCliente, d.domicilioCliente, " +
        "c.NombreCliente, c.RFCCliente " +
        "FROM Comercial.DomicilioCliente d " +
        "JOIN Comercial.Cliente c ON d.idCliente = c.idCliente " + // Agregamos un espacio al final de esta línea
        "WHERE c.idCliente = @IdCliente";

```

```

SqlCommand cmd = new SqlCommand(selectInfo, nuevaConexion.AbrirConexion());

cmd.Parameters.AddWithValue("@IdCliente", idCliente);

SqlDataReader reader = cmd.ExecuteReader();

// Agrega columnas en la TablaDomicilioCliente si es que no contiene columnas.
if (TablaDomicilioCliente.Columns.Count == 0)
{
    TablaDomicilioCliente.Columns.Add("idDomicilio", "ID");
    TablaDomicilioCliente.Columns.Add("idCliente", "Cliente");
    TablaDomicilioCliente.Columns.Add("domicilioCliente", "Domicilio");
}

TablaDomicilioCliente.Rows.Clear();

int i = 0;

// Agrega a TablaDomicilioCliente una nueva fila para almacenar con los valores correspondientes a las columnas.
while (reader.Read())
{
    string nombreCliente = reader["NombreCliente"].ToString();
    string rfcCliente = reader["RFCCliente"].ToString();
    string cliente = $"{nombreCliente}-{rfcCliente.Substring(rfcCliente.Length - 3)}";

    TablaDomicilioCliente.Rows.Add();
    TablaDomicilioCliente.Rows[i].Cells[0].Value = reader["idDomicilio"].ToString();
    TablaDomicilioCliente.Rows[i].Cells[1].Value = cliente;
    TablaDomicilioCliente.Rows[i].Cells[2].Value = reader["domicilioCliente"].ToString();
    i++;
}

reader.Close();
nuevaConexion.CerrarConexion();
}

/*
TablaDomicilioCliente_CellClick

Evento que se activa al hacer clic en una celda del DataGridView.

Llena los campos de entrada con los datos de la fila seleccionada.

*/

```

```

private void TablaDomicilioCliente_CellClick(object sender, DataGridViewCellEventArgs e)
{
    if (e.RowIndex >= 0)
    {
        DataGridViewRow row = TablaDomicilioCliente.Rows[e.RowIndex];

        idDomicilio = int.Parse(row.Cells[0].Value.ToString());

        txtDomCliente.Text = row.Cells[2].Value.ToString();
    }
}

/*
    Verifica si un domicilio específico ya existe para un cliente.
*/
public bool DomicilioExiste(string domicilio)
{
    nuevaConexion.AbrirConexion();

    string query = "SELECT COUNT(*) FROM Comercial.DomicilioCliente WHERE idCliente = @IdCliente AND domicilioCliente = @Domicilio";

    SqlCommand cmd = new SqlCommand(query, nuevaConexion.AbrirConexion());

    cmd.Parameters.AddWithValue("@IdCliente", idCliente);

    cmd.Parameters.AddWithValue("@Domicilio", domicilio);

    int count = (int)cmd.ExecuteScalar();

    nuevaConexion.CerrarConexion();

    return count > 0;
}

/*
    InsertaDato

    Inserta un nuevo registro en la tabla Comercial.DomicilioCliente con los datos ingresados.

    Si ocurre un error (como duplicación de un domicilio), muestra un mensaje de error.
*/
public void InsertaDato()
{
    string domicilioCliente = txtDomCliente.Text;

```

```

        if (DomicilioExiste(domicilioCliente))
        {
            MessageBox.Show("El domicilio ya existe para este cliente.", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
            return;
        }

        nuevaConexion.AbrirConexion();

        // Cadena SQL para agregar un nuevo registro en la tabla Comercial.DomicilioCliente.
        string InsertInfo = "INSERT INTO Comercial.DomicilioCliente(idCliente, domicilioCliente)" +
            "VALUES(@IdCliente, @DomCliente)";
        SqlCommand cm = new SqlCommand(InsertInfo, nuevaConexion.AbrirConexion());
        cm.Parameters.AddWithValue("@IdCliente", idCliente);
        cm.Parameters.AddWithValue("@DomCliente", domicilioCliente);
        cm.ExecuteNonQuery();
        nuevaConexion.CerrarConexion();
    }

    /*
    ModificaDato
    Modifica el registro seleccionado en la tabla Comercial.DomicilioCliente con los datos actualizados.
    Si ocurre un error (como duplicación de un domicilio), muestra un mensaje de error.
    */
    public void ModificaDato()
    {
        string domicilioCliente = txtDomCliente.Text;

        if (DomicilioExiste(domicilioCliente))
        {
            MessageBox.Show("El domicilio ya existe para este cliente.", "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
            return;
        }

        nuevaConexion.AbrirConexion();

        // Cadena SQL para modificar un nuevo registro en la tabla Comercial.DomicilioCliente.

```

```
string update = "UPDATE Comercial.DomicilioCliente SET idCliente = @idCliente, domicilioCliente = @DomicilioCliente  
WHERE idDomicilio = @Id";
```

```
SqlCommand cmd = new SqlCommand(update, nuevaConexion.AbrirConexion());  
  
cmd.Parameters.AddWithValue("@idCliente", idCliente);  
  
cmd.Parameters.AddWithValue("@domicilioCliente", domicilioCliente);  
  
cmd.Parameters.AddWithValue("@Id", idDomicilio);  
  
cmd.ExecuteNonQuery();  
  
nuevaConexion.CerrarConexion();  
  
}
```

```
/*
```

EliminaDato

Elimina el registro seleccionado en la tabla Comercial.DomicilioCliente con los datos actualizados.

Si existe un servicio ya asociado a dicho domicilio, muestra un mensaje de error.

```
*/
```

```
public void EliminaDato()
```

```
{
```

```
try
```

```
{
```

```
if (idDomicilio != -1)
```

```
{
```

```
nuevaConexion.AbrirConexion();
```

```
// Cadena SQL para eliminar un nuevo registro en la tabla Comercial.DomicilioCliente.
```

```
string delete = "DELETE FROM Comercial.DomicilioCliente WHERE idDomicilio = @Id";
```

```
SqlCommand cmd = new SqlCommand(delete, nuevaConexion.AbrirConexion());
```

```
cmd.Parameters.AddWithValue("@Id", idDomicilio);
```

```
cmd.ExecuteNonQuery();
```

```
string checkEmptyQuery = "SELECT COUNT(*) FROM Comercial.DomicilioCliente";
```

```
SqlCommand checkCmd = new SqlCommand(checkEmptyQuery, nuevaConexion.AbrirConexion());
```

```
int rowCount = (int)checkCmd.ExecuteScalar();
```

```
// Reinicia el valor del idCliente cuando la cantidad de filas es igual a 0.
```

```
if (rowCount == 0)
```

```
{
```

```
string resetIdentityQuery = "DBCC CHECKIDENT ('Comercial.DomicilioCliente', RESEED, 0)";
```

```

        SqlCommand resetCmd = new SqlCommand(resetIdentityQuery, nuevaConexion.AbrirConexion());

        resetCmd.ExecuteNonQuery();

    }

    nuevaConexion.CerrarConexion();

}

catch

{

    MessageBox.Show("Existe al menos un servicio asociado a dicho domicilio.");

}

}

/*

    btnInserta_Click

    Evento de clic del botón de inserción.

    Llama a InsertaDato, actualiza el DataGridView llamando a ConsultaDatos y limpia los campos de entrada.

    */

private void btnInserta_Click(object sender, EventArgs e)

{

    InsertaDato();

    ConsultaDatos();

    txtDomCliente.Clear();

}

/*

    btnModifica_Click

    Evento de clic del botón de modificación.

    Llama a ModificaDato, actualiza el DataGridView llamando a ConsultaDatos y limpia los campos de entrada.

    */

private void btnModifica_Click(object sender, EventArgs e)

{

    ModificaDato();

    ConsultaDatos();

    txtDomCliente.Clear();

}

/*

```

```

        btnElimina_Click

        Evento de clic del botón de eliminación.

        Llama a EliminaDato, actualiza el DataGridView llamando a ConsultaDatos y limpia los campos de entrada.

        */

private void btnElimina_Click(object sender, EventArgs e)
{
    EliminaDato();

    ConsultaDatos();

    txtDomCliente.Clear();
}
}
}

```

Formulario Proveedor.

Código documentado del Formulario Proveedor.

// Librerías del sistema y .NET Framework usadas para crear y gestionar el formulario Proveedor.

```

using System;

using System.Collections.Generic;

using System.ComponentModel;

using System.Data;

using System.Data.SqlClient;

using System.Drawing;

using System.Linq;

using System.Text;

using System.Threading.Tasks;

using System.Windows.Forms;

```

```

namespace ServiciosDeMantenimiento

```



```

{
    /*
        Clase para gestionar el manejo de los proveedores dentro de los Servicios de Mantenimiento
        Proporciona métodos para consultar, insertar, modificar y eliminar datos.
    */

    public partial class Proveedor : Form
    {
        // Crea una instancia de una nueva conexión para conectarse con la base de datos de SQL Server.
        ConexionDB nuevaConexion = new ConexionDB("Server=ASUS\\SQLEXPRESS1; DATABASE = ServiciosdeMantenimiento; " +
            "integrated security = true");

        int idProveedor = -1;
        string nombreProveedor = "";
        string telefonoProveedor = "";
        string emailProveedor = "";

        /*
            Constructor del formulario Proveedor.
            Inicializa el formulario y carga los datos de la tabla Proveedor en el DataGridView.
            También suscribe el evento CellClick de la tabla.
        */

        public Proveedor()
        {
            InitializeComponent();
            ConsultaDatos();
            TablaProveedor.CellClick += TablaProveedor_CellClick;
        }

        /*
            ConsultaDatos
            Recupera y muestra todos los registros de la tabla Comercial.DomicilioCliente en el DataGridView.
            Formatea las columnas y limpia las filas antes de agregar los datos.
        */

        public void ConsultaDatos()
        {
            nuevaConexion.AbrirConexion();
            TablaProveedor.DataSource = null;

```

```

// Cadena SQL para consultar la tabla Comercial.Proveedor.

string selectInfo = "SELECT * FROM Comercial.Proveedor";

SqlCommand cmd = new SqlCommand(selectInfo, nuevaConexion.AbrirConexion());

SqlDataReader reader = cmd.ExecuteReader();


// Agrega columnas en la TablaProveedor si es que no contiene columnas.
if (TablaProveedor.Columns.Count == 0)
{
    TablaProveedor.Columns.Add("idProveedor", "ID");

    TablaProveedor.Columns.Add("nombreProveedor", "Nombre del proveedor");

    TablaProveedor.Columns.Add("telefonoProveedor", "Teléfono del proveedor");

    TablaProveedor.Columns.Add("emailProveedor", "Email del proveedor");

}

TablaProveedor.Rows.Clear();

int i = 0;


// Agrega a TablaProveedor una nueva fila para almacenar con los valores correspondientes a las columnas.
while (reader.Read())
{
    TablaProveedor.Rows.Add();

    TablaProveedor.Rows[i].Cells[0].Value = reader["idProveedor"].ToString();

    TablaProveedor.Rows[i].Cells[1].Value = reader["nombreProveedor"].ToString();

    TablaProveedor.Rows[i].Cells[2].Value = reader["telefonoProveedor"].ToString();

    TablaProveedor.Rows[i].Cells[3].Value = reader["emailProveedor"].ToString();

    i++;

}

reader.Close();

nuevaConexion.CerrarConexion();

}

/*
TablaProveedor_CellClick

Evento que se activa al hacer clic en una celda del DataGridView.

Llena los campos de entrada con los datos de la fila seleccionada.

*/

private void TablaProveedor_CellClick(object sender, DataGridViewCellEventArgs e)

```

```

{
    if (e.RowIndex >= 0)
    {
        DataGridViewRow row = TablaProveedor.Rows[e.RowIndex];

        idProveedor = int.Parse(row.Cells[0].Value.ToString());

        txtNombre_Proveedor.Text = row.Cells[1].Value.ToString();

        txtTelefono_Proveedor.Text = row.Cells[2].Value.ToString();

        txtEmail_Proveedor.Text = row.Cells[3].Value.ToString();

    }
}

/*
    InsertaDato

    Inserta un nuevo registro en la tabla Comercial.Proveedor con los datos ingresados.

    Si ocurre un error (como duplicación de un teléfono o un correo electrónico), muestra un mensaje de error.
*/

public void InsertaDato()
{
    nombreProveedor = txtNombre_Proveedor.Text;

    telefonoProveedor = txtTelefono_Proveedor.Text;

    emailProveedor = txtEmail_Proveedor.Text;

    nuevaConexion.AbrirConexion();

    //Verifica si el teléfono o el correo electrónico ya existen.

    string verifica = "SELECT COUNT(*) FROM Comercial.Proveedor WHERE telefonoProveedor = @Telefono OR emailProveedor = @Email";

    SqlCommand checkCmd = new SqlCommand(verifica, nuevaConexion.AbrirConexion());

    checkCmd.Parameters.AddWithValue("@Telefono", telefonoProveedor);

    checkCmd.Parameters.AddWithValue("@Email", emailProveedor);

    int count = (int)checkCmd.ExecuteScalar();

    if (count > 0)
    {
        MessageBox.Show("El teléfono o el correo electrónico ya están en uso.");

    }

    else

```

```

{
    // Cadena SQL para agregar un nuevo registro en la tabla Comercial.Proveedor.
    string InsertInfo = "INSERT INTO Comercial.Proveedor(nombreProveedor, telefonoProveedor, emailProveedor) " +
        "VALUES (@Nombre, @Telefono, @Email)";

    SqlCommand cm = new SqlCommand(InsertInfo, nuevaConexion.AbrirConexion());
    cm.Parameters.AddWithValue("@Nombre", nombreProveedor);
    cm.Parameters.AddWithValue("@Telefono", telefonoProveedor);
    cm.Parameters.AddWithValue("@Email", emailProveedor);
    cm.ExecuteNonQuery();
}

nuevaConexion.CerrarConexion();
}

/*
    Modifica el registro seleccionado en la tabla COmercial.Proveedor con los datos actualizados.
    Si ocurre un error (como duplicación de un teléfono o un correo electrónico), muestra un mensaje de error.
*/

public void ModificaDato()
{
    nombreProveedor = txtNombre_Proveedor.Text;
    telefonoProveedor = txtTelefono_Proveedor.Text;
    emailProveedor = txtEmail_Proveedor.Text;

    nuevaConexion.AbrirConexion();

    //Verifica si el teléfono o el correo electrónico ya existen.
    string verifica = "SELECT COUNT(*) FROM Comercial.Proveedor WHERE (telefonoProveedor = @Telefono OR emailProveedor = @Email) AND idProveedor != @Id";

    SqlCommand checkCmd = new SqlCommand(verifica, nuevaConexion.AbrirConexion());
    checkCmd.Parameters.AddWithValue("@Telefono", telefonoProveedor);
    checkCmd.Parameters.AddWithValue("@Email", emailProveedor);
    checkCmd.Parameters.AddWithValue("@Id", idProveedor);
    int count = (int)checkCmd.ExecuteScalar();

    if (count > 0)
    {

```

```

        MessageBox.Show("El teléfono o el correo electrónico ya están en uso por otro proveedor.");
    }
    else
    {
        // Cadena SQL para modificar un nuevo registro en la tabla Comercial.Proveedor.

        string update = "UPDATE Comercial.Proveedor SET nombreProveedor = @Nombre, telefonoProveedor = @Telefono,
emailProveedor = @Email " +

            "WHERE idProveedor = @Id";

        SqlCommand cmd = new SqlCommand(update, nuevaConexion.AbrirConexion());

        cmd.Parameters.AddWithValue("@Nombre", nombreProveedor);

        cmd.Parameters.AddWithValue("@Telefono", telefonoProveedor);

        cmd.Parameters.AddWithValue("@Email", emailProveedor);

        cmd.Parameters.AddWithValue("@Id", idProveedor);

        cmd.ExecuteNonQuery();

    }

    nuevaConexion.CerrarConexion();
}

/*
EliminaDato

Elimina el registro seleccionado en la tabla Comercial.Proveedor con los datos actualizados.
*/

public void EliminaDato()
{
    if (idProveedor != -1)
    {
        nuevaConexion.AbrirConexion();

        // Cadena SQL para eliminar un nuevo registro en la tabla Comercial.Proveedor.

        string delete = "DELETE FROM Comercial.Proveedor WHERE idProveedor = @Id";

        SqlCommand cmd = new SqlCommand(delete, nuevaConexion.AbrirConexion());

        cmd.Parameters.AddWithValue("@Id", idProveedor);

        cmd.ExecuteNonQuery();

        string checkEmptyQuery = "SELECT COUNT(*) FROM Comercial.Proveedor";

        SqlCommand checkCmd = new SqlCommand(checkEmptyQuery, nuevaConexion.AbrirConexion());

```

```

int rowCount = (int)checkCmd.ExecuteScalar();

// Reinicia el valor del idProveedor cuando la cantidad de filas es igual a 0.
if (rowCount == 0)
{
    string resetIdentityQuery = "DBCC CHECKIDENT ('Comercial.Proveedor', RESEED, 0)";
    SqlCommand resetCmd = new SqlCommand(resetIdentityQuery, nuevaConexion.AbrirConexion());
    resetCmd.ExecuteNonQuery();
}
nuevaConexion.CerrarConexion();
}
}

/*
btnInserta_Click

Evento de clic del botón de inserción.

Llama a InsertaDato, actualiza el DataGridView llamando a ConsultarDatos y limpia los campos de entrada.
*/

private void btnInserta_Click(object sender, EventArgs e)
{
    InsertaDato();
    ConsultarDatos();
    txtNombre_Proveedor.Clear();
    txtTelefono_Proveedor.Clear();
    txtEmail_Proveedor.Clear();
    idProveedor = -1;
}

/*
btnModifica_Click

Evento de clic del botón de modificación.

Llama a ModificaDato, actualiza el DataGridView llamando a ConsultarDatos y limpia los campos de entrada.
*/

private void btnModifica_Click(object sender, EventArgs e)
{
    ModificaDato();
    ConsultarDatos();
}

```

```

txtNombre_Proveedor.Clear();

txtTelefono_Proveedor.Clear();

txtEmail_Proveedor.Clear();

idProveedor = -1;
}

/*
btnElimina_Click

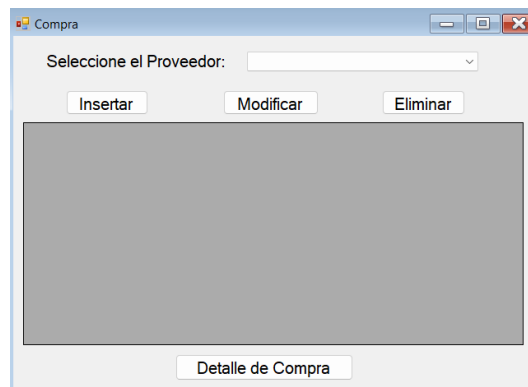
Evento de clic del botón de eliminación.

Llama a EliminaDato, actualiza el DataGridView llamando a ConsultaDatos y limpia los campos de entrada.
*/

private void btnElimina_Click(object sender, EventArgs e)
{
    EliminaDato();
    ConsultaDatos();
    txtNombre_Proveedor.Clear();
    txtTelefono_Proveedor.Clear();
    txtEmail_Proveedor.Clear();
    idProveedor = -1;
}
}
}

```

Formulario Compra.



Código documentado del Formulario Compra.

```

// Librerías del sistema y .NET Framework usadas para crear y gestionar el formulario Compra.

using System;

using System.Collections.Generic;

```

```

using System.ComponentModel;
using System.Data;
using System.Data.SqlClient;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace ServiciosDeMantenimiento
{
    /*
        Clase para gestionar el manejo de las compras dentro de los Servicios de Mantenimiento
        Proporciona métodos para consultar, insertar, modificar y eliminar datos.
    */
    public partial class Compra : Form
    {
        // Crea una instancia de una nueva conexión para conectarse con la base de datos de SQL Server.
        ConexionDB nuevaConexion = new ConexionDB("Server=ASUS\\SQLEXPRESS1; DATABASE = ServiciosdeMantenimiento; " +
            "integrated security = true");

        int idCompra = -1;
        string idProveedor = "";
        string fechaCompra = "";
        string totalCompra = "0.00";

        //Lista que almacena objetos de tipo DetalleCompra.
        private List<DetalleCompra> detalleCompraForms = new List<DetalleCompra>();

        /*
            Constructor del formulario Compra.
            Inicializa el formulario y carga los datos de la tabla Compra en el DataGridView.
            También suscribe el evento CellClick de la tabla.
        */
        public Compra()
        {
            InitializeComponent();
            ConsultaDatos();
        }
    }

```



```

    CargarProveedores();

    TablaCompra.CellClick += TablaCompra_CellClick;
}

/*
    ConsultaDatos

    Recupera y muestra todos los registros de la tabla Comercial.Compra en el DataGridView.

    Formatea la columna de precios y limpia las filas antes de agregar los datos.
*/
public void ConsultaDatos()
{
    nuevaConexion.AbrirConexion();

    TablaCompra.DataSource = null;

    // Cadena SQL para consultar la tabla Comercial.Compra.
    string selectInfo = "SELECT c.idCompra, c.idProveedor, c.fechaCompra, c.totalCompra, " +
        "p.NombreProveedor, p.TelefonoProveedor " +
        "FROM Comercial.Compra c " +
        "JOIN Comercial.Proveedor p ON c.idProveedor = p.idProveedor";

    SqlCommand cmd = new SqlCommand(selectInfo, nuevaConexion.AbrirConexion());

    SqlDataReader reader = cmd.ExecuteReader();

    // Agrega columnas en la TablaCompra si es que no contiene columnas.
    if (TablaCompra.Columns.Count == 0)
    {
        TablaCompra.Columns.Add("idCompra", "ID");

        TablaCompra.Columns.Add("idProveedor", "Proveedor");

        TablaCompra.Columns.Add("fechaCompra", "Fecha de la compra");

        TablaCompra.Columns.Add("totalCompra", "Total de la compra");

        TablaCompra.Columns["totalCompra"].DefaultCellStyle.Format = "N2";
    }

    TablaCompra.Rows.Clear();

    int i = 0;

    // Agrega a TablaCompra una nueva fila para almacenar con los valores correspondientes a las columnas.

```

```

while (reader.Read())
{
    string nombreProveedor = reader["NombreProveedor"].ToString();
    string telefonoProveedor = reader["TelefonoProveedor"].ToString();
    string proveedor = $"{nombreProveedor} - {telefonoProveedor.Substring(telefonoProveedor.Length - 3)}";

    TablaCompra.Rows.Add();
    TablaCompra.Rows[i].Cells[0].Value = reader["idCompra"].ToString();
    TablaCompra.Rows[i].Cells[1].Value = proveedor; // Aquí se muestra la concatenación
    TablaCompra.Rows[i].Cells[2].Value = Convert.ToDateTime(reader["fechaCompra"]).ToString("dd-MM-yyyy");
    var totalValue = reader["totalCompra"];
    if (totalValue == DBNull.Value || string.IsNullOrEmpty(totalValue.ToString()))
    {
        TablaCompra.Rows[i].Cells[3].Value = "0.00";
    }
    else
    {
        TablaCompra.Rows[i].Cells[3].Value = Convert.ToDecimal(totalValue).ToString("N2");
    }
    i++;
}
reader.Close();
nuevaConexion.CerrarConexion();
}

/*
    Carga la información de la tabla Comercial.Proveedor y la muestra en un combobox.
*/
public void CargarProveedores()
{
    nuevaConexion.AbrirConexion();
    string selectInfo = "SELECT NombreProveedor, TelefonoProveedor FROM Comercial.Proveedor";

    SqlCommand cmd = new SqlCommand(selectInfo, nuevaConexion.AbrirConexion());
    SqlDataReader reader = cmd.ExecuteReader();

    cmb_Proveedor.Items.Clear();

```

```

// Lee cada proveedor del resultado de la consulta
while (reader.Read())
{
    string nombreProveedor = reader["NombreProveedor"].ToString();
    string telefonoProveedor = reader["TelefonoProveedor"].ToString();
    string proveedor = $"{nombreProveedor} - {telefonoProveedor.Substring(telefonoProveedor.Length - 3)}";

    // Agrega el resultado al combobox
    cmb_Proveedor.Items.Add(proveedor);
}

reader.Close();
nuevaConexion.CerrarConexion();
}

/*
TablaCompra_CellClick

Evento que se activa al hacer clic en una celda del DataGridView.

Llena los campos de entrada con los datos de la fila seleccionada.
*/
private void TablaCompra_CellClick(object sender, DataGridViewCellEventArgs e)
{
    if (e.RowIndex >= 0)
    {
        DataGridViewRow row = TablaCompra.Rows[e.RowIndex];
        idCompra = int.Parse(row.Cells[0].Value.ToString());
        cmb_Proveedor.Text = row.Cells[1].Value.ToString();

        // Hace visible al botón btnDetalleCompra.
        btnDetalleCompra.Visible = true;
    }
}

/*
InsertaDato

Inserta un nuevo registro en la tabla Comercial.Compra con los datos ingresados.

```

```

*/

public void InsertaDato()
{
    string proveedorSeleccionado = cmb_Proveedor.Text;

    string[] partesProveedor = proveedorSeleccionado.Split('-');

    string nombreProveedor = partesProveedor[0].Trim();

    string ultimosTresTelefono = partesProveedor[1].Trim();

    nuevaConexion.AbrirConexion();

    //Rescatar la concatenación del proveedor.

    string queryIdProveedor = "SELECT idProveedor FROM Comercial.Proveedor WHERE NombreProveedor = @Nombre AND RIGHT(TelefonoProveedor, 3) = @UltimosTres";

    SqlCommand cmdIdProveedor = new SqlCommand(queryIdProveedor, nuevaConexion.AbrirConexion());

    cmdIdProveedor.Parameters.AddWithValue("@Nombre", nombreProveedor);

    cmdIdProveedor.Parameters.AddWithValue("@UltimosTres", ultimosTresTelefono);

    idProveedor = cmdIdProveedor.ExecuteScalar().ToString();

    nuevaConexion.CerrarConexion();

    DateTime fechaCompra = DateTime.Now;

    nuevaConexion.AbrirConexion();

    // Cadena SQL para agregar un nuevo registro en la tabla Comercial.Compra.

    string InsertInfo = "INSERT INTO Comercial.Compra(idProveedor, fechaCompra) " +

        "VALUES (@IdProveedor, @Fecha)";

    SqlCommand cm = new SqlCommand(InsertInfo, nuevaConexion.AbrirConexion());

    cm.Parameters.AddWithValue("@IdProveedor", idProveedor);

    cm.Parameters.AddWithValue("@Fecha", fechaCompra);

    cm.ExecuteNonQuery();

    nuevaConexion.CerrarConexion();

}

/*

ModificaDato

Modifica el registro seleccionado en la tabla Comercial.Compra con los datos actualizados.

```

```

*/

public void ModificaDato()
{
    string proveedorSeleccionado = cmb_Proveedor.Text;

    string[] partesProveedor = proveedorSeleccionado.Split('-');

    string nombreProveedor = partesProveedor[0].Trim();

    string ultimosTresTelefono = partesProveedor[1].Trim();

    // Obtén el idProveedor de la base de datos basado en el nombre del proveedor y los últimos 3 dígitos del teléfono

    nuevaConexion.AbrirConexion();

    string queryIdProveedor = "SELECT idProveedor FROM Comercial.Proveedor WHERE NombreProveedor = @Nombre AND RIGHT(TelefonoProveedor, 3) = @UltimosTres";

    SqlCommand cmdIdProveedor = new SqlCommand(queryIdProveedor, nuevaConexion.AbrirConexion());

    cmdIdProveedor.Parameters.AddWithValue("@Nombre", nombreProveedor);

    cmdIdProveedor.Parameters.AddWithValue("@UltimosTres", ultimosTresTelefono);

    idProveedor = cmdIdProveedor.ExecuteScalar().ToString();

    nuevaConexion.CerrarConexion();

    nuevaConexion.AbrirConexion();

    // Cadena SQL para modificar un nuevo registro en la tabla Comercial.Compra.

    string update = "UPDATE Comercial.Compra SET idProveedor = @idProveedor WHERE idCompra = @Id";

    SqlCommand cmd = new SqlCommand(update, nuevaConexion.AbrirConexion());

    cmd.Parameters.AddWithValue("@idProveedor", idProveedor);

    cmd.Parameters.AddWithValue("@Id", idCompra);

    cmd.ExecuteNonQuery();

    nuevaConexion.CerrarConexion();
}

/*

EliminaDato

Elimina el registro seleccionado en la tabla Comercial.Compra con los datos actualizados.

*/

public void EliminaDato()
{
    if (idCompra != -1)
    {

```

```

nuevaConexion.AbrirConexion();

// Eliminar primero los detalles de la compra.
string deleteDetalles = "DELETE FROM Comercial.DetalleCompra WHERE idCompra = @Id";
SqlCommand cmdDetalles = new SqlCommand(deleteDetalles, nuevaConexion.AbrirConexion());
cmdDetalles.Parameters.AddWithValue("@Id", idCompra);
cmdDetalles.ExecuteNonQuery();

// Cadena SQL para eliminar un nuevo registro en la tabla Comercial.Compra.
string delete = "DELETE FROM Comercial.Compra WHERE idCompra = @Id";
SqlCommand cmd = new SqlCommand(delete, nuevaConexion.AbrirConexion());
cmd.Parameters.AddWithValue("@Id", idCompra);
cmd.ExecuteNonQuery();

string checkEmptyQuery = "SELECT COUNT(*) FROM Comercial.Compra";
SqlCommand checkCmd = new SqlCommand(checkEmptyQuery, nuevaConexion.AbrirConexion());
int rowCount = (int)checkCmd.ExecuteScalar();

// Reinicia el valor del idCompra cuando la cantidad de filas es igual a 0.
if (rowCount == 0)
{
    string resetIdentityQuery = "DBCC CHECKIDENT ('Comercial.Compra', RESEED, 0);";
    SqlCommand resetCmd = new SqlCommand(resetIdentityQuery, nuevaConexion.AbrirConexion());
    resetCmd.ExecuteNonQuery();
}

nuevaConexion.CerrarConexion();
}
}

/*
Recupera el valor de la columna del idCompra.
*/

private int ObtenerIdCompra()
{
    if (TablaCompra.SelectedRows.Count > 0)
    {
        DataGridViewRow row = TablaCompra.SelectedRows[0];
    }
}

```

```

        int idCompra = int.Parse(row.Cells[0].Value.ToString()); // Asegúrate de que el índice de la celda sea correcto

        return idCompra;
    }

    throw new InvalidOperationException("No hay una compra seleccionada.");
}

/*
    btnInserta_Click

    Evento de clic del botón de inserción.

    Llama a InsertaDato, actualiza el DataGridView llamando a ConsultarDatos y limpia los campos de entrada.
*/
private void btnInserta_Click(object sender, EventArgs e)
{
    InsertaDato();
    ConsultarDatos();
    cmb_Proveedor.SelectedIndex = -1;
}

/*
    btnModifica_Click

    Evento de clic del botón de modificación.

    Llama a ModificaDato, actualiza el DataGridView llamando a ConsultarDatos y limpia los campos de entrada.
*/
private void btnModifica_Click(object sender, EventArgs e)
{
    ModificaDato();
    ConsultarDatos();
    cmb_Proveedor.SelectedIndex = -1;
}

/*
    btnElimina_Click

    Evento de clic del botón de eliminación.

    Llama a EliminaDato, actualiza el DataGridView llamando a ConsultarDatos y limpia los campos de entrada.
*/
private void btnElimina_Click(object sender, EventArgs e)

```

```

{
    EliminaDato();

    ConsultaDatos();

    cmb_Proveedor.SelectedIndex = -1;
}

/*
    Evento que se dispara al hacer clic en el botón de Detalle de Compra.

    Abre el formulario DetalleCompra para gestionar los detalles asociados a una compra específica.
*/
private void btnDetalleCompra_Click(object sender, EventArgs e)
{
    if (idCompra != -1)
    {
        DetalleCompra detalleCompraForm = new DetalleCompra(idCompra);
        detalleCompraForms.Add(detalleCompraForm);
        detalleCompraForm.SubtotalChanged += DetalleCompraForm_SubtotalChanged;
        detalleCompraForm.FormClosed += (s, args) =>
        {
            detalleCompraForms.Remove(detalleCompraForm);
        };
        detalleCompraForm.Show();
    }
}

/*
    Ejecuta el Evento el cual es disparado por una instancia de DetalleCompra.
*/
private void DetalleCompraForm_SubtotalChanged(object sender, EventArgs e)
{
    ConsultaDatos();
}
}

```


Formulario DetalleCompra.

Código documentado del Formulario DetalleCompra.

// Librerías del sistema y .NET Framework usadas para crear y gestionar el formulario DetalleCompra.

```
using System;
```

```
using System.Collections.Generic;
```

```
using System.ComponentModel;
```

```
using System.Data;
```

```
using System.Data.SqlClient;
```

```
using System.Drawing;
```

```
using System.Linq;
```

```
using System.Text;
```

```
using System.Threading.Tasks;
```

```
using System.Windows.Forms;
```

```
namespace ServiciosDeMantenimiento
```

```
{
```

```
    /*
```

```
        Clase para gestionar el manejo de los detalles de las compras dentro de los Servicios de Mantenimiento
```

```
        Proporciona métodos para consultar, insertar, modificar y eliminar datos.
```

```
    */
```

```
    public partial class DetalleCompra : Form
```

```
    {
```

```
        // Crea una instancia de una nueva conexión para conectarse con la base de datos de SQL Server.
```

```
        ConexionDB nuevaConexion = new ConexionDB("Server=ASUS\\SQLEXPRESS1; DATABASE = ServiciosdeMantenimiento; " +  
            "integrated security = true");
```

```
        int idCompra;
```

```
        string idMaterial = "";
```

```
string cantidad;
```

```
float subtotal;
```

```
// Define un evento llamado SubtotalChanged el cual se utiliza para notificar cuando ocurre un cambio en el subtotal de una compra.
```

```
public event EventHandler SubtotalChanged;
```

```
/*
```

```
Constructor del formulario DetalleCompra.
```

```
Inicializa el formulario y carga los datos de la tabla DetalleCompra en el DataGridView.
```

```
También suscribe el evento CellClick de la tabla.
```

```
*/
```

```
public DetalleCompra(int idCompra)
```

```
{
```

```
InitializeComponent();
```

```
this.idCompra = idCompra;
```

```
ConsultaDatos();
```

```
CargarMateriales();
```

```
TablaDetalleCompra.CellClick += TablaDetalleCompra_CellClick;
```

```
}
```

```
/*
```

```
ConsultaDatos
```

```
Recupera y muestra todos los registros de la tabla Comercial.DetalleCompra en el DataGridView.
```

```
Formatea las columnas y limpia las filas antes de agregar los datos.
```

```
*/
```

```
public void ConsultaDatos()
```

```
{
```

```
nuevaConexion.AbrirConexion();
```

```
TablaDetalleCompra.DataSource = null;
```

```
// Cadena SQL para consultar la tabla Comercial.DetalleCompra.
```

```
string selectInfo = "SELECT d.idCompra, d.idMaterial, d.cantidad, d.subtotal, " +
```

```
"m.NombreMaterial, m.TipoMaterial " +
```

```
"FROM Comercial.DetalleCompra d " +
```

```
"JOIN Operativo.Material m ON d.idMaterial = m.idMaterial " + // Notice the space added here
```

```
"WHERE d.idCompra = @IdCompra";
```

```

SqlCommand cmd = new SqlCommand(selectInfo, nuevaConexion.AbrirConexion());

cmd.Parameters.AddWithValue("@IdCompra", idCompra);

SqlDataReader reader = cmd.ExecuteReader();

// Agrega columnas en la TablaDetalleCompra si es que no contiene columnas.
if (TablaDetalleCompra.Columns.Count == 0)
{
    TablaDetalleCompra.Columns.Add("idCompra", "Compra");
    TablaDetalleCompra.Columns.Add("idMaterial", "Material");
    TablaDetalleCompra.Columns.Add("cantidad", "Cantidad");
    TablaDetalleCompra.Columns.Add("subtotal", "subtotal");

    TablaDetalleCompra.Columns["subtotal"].DefaultCellStyle.Format = "N2";
}

TablaDetalleCompra.Rows.Clear();

int i = 0;

// Agrega a TablaDetalleCompra una nueva fila para almacenar con los valores correspondientes a las columnas.
while (reader.Read())
{
    string nombreMaterial = reader["NombreMaterial"].ToString();
    string tipoMaterial = reader["TipoMaterial"].ToString();
    string material = $"{nombreMaterial} - {tipoMaterial}";

    TablaDetalleCompra.Rows.Add();
    TablaDetalleCompra.Rows[i].Cells[0].Value = reader["idCompra"].ToString();
    TablaDetalleCompra.Rows[i].Cells[1].Value = material;
    TablaDetalleCompra.Rows[i].Cells[2].Value = Convert.ToInt64(reader["cantidad"]).ToString();
    TablaDetalleCompra.Rows[i].Cells[3].Value = Convert.ToDecimal(reader["subtotal"]).ToString("N2");

    i++;
}

reader.Close();

nuevaConexion.CerrarConexion();
}

```

```

/*
    Carga la información de la tabla Operativo.Material y la muestra en un combobox.
*/
public void CargarMateriales()
{
    nuevaConexion.AbrirConexion();

    string selectInfo = "SELECT NombreMaterial, TipoMaterial FROM Operativo.Material";

    SqlCommand cmd = new SqlCommand(selectInfo, nuevaConexion.AbrirConexion());
    SqlDataReader reader = cmd.ExecuteReader();

    cmb_Material.Items.Clear();

    // Lee cada material del resultado de la consulta
    while (reader.Read())
    {
        string nombreMaterial = reader["NombreMaterial"].ToString();
        string tipoMaterial = reader["TipoMaterial"].ToString();
        string material = $"{nombreMaterial} - {tipoMaterial}";

        // Agrega el resultado al combo box
        cmb_Material.Items.Add(material);
    }

    reader.Close();
    nuevaConexion.CerrarConexion();
}

/*
    TablaDetalleCompra_CellClick

    Evento que se activa al hacer clic en una celda del DataGridView.

    Llena los campos de entrada con los datos de la fila seleccionada.
*/
private void TablaDetalleCompra_CellClick(object sender, DataGridViewCellEventArgs e)
{
    if (e.RowIndex >= 0)
    {

```

```

        DataGridViewRow row = TablaDetalleCompra.Rows[e.RowIndex];

        txtCantidad.Text = row.Cells[2].Value.ToString();

        cmb_Material.Text = row.Cells[1].Value.ToString();
    }
}

/*
InsertaDato

Inserta un nuevo registro en la tabla Comercial.DetalleCompra con los datos ingresados.
*/

public void InsertaDato()
{
    cantidad = txtCantidad.Text;

    string materialSeleccionado = cmb_Material.Text;

    string[] partesMaterial = materialSeleccionado.Split('-');

    string nombreMaterial = partesMaterial[0].Trim();

    string tipoMaterial = partesMaterial[1].Trim();

    nuevaConexion.AbrirConexion();

    //Rescatar la concatenación del material.

    string queryIdMaterial = "SELECT idMaterial FROM Operativo.Material WHERE NombreMaterial = @Nombre AND TipoMaterial = @Tipo";

    SqlCommand cmdIdMaterial = new SqlCommand(queryIdMaterial, nuevaConexion.AbrirConexion());

    cmdIdMaterial.Parameters.AddWithValue("@Nombre", nombreMaterial);

    cmdIdMaterial.Parameters.AddWithValue("@Tipo", tipoMaterial);

    idMaterial = cmdIdMaterial.ExecuteScalar()?.ToString();

    nuevaConexion.CerrarConexion();

    nuevaConexion.AbrirConexion();

    // Cadena SQL para agregar un nuevo registro en la tabla Comercial.DetalleCompra.

    string InsertInfo = "INSERT INTO Comercial.DetalleCompra(idCompra, idMaterial, cantidad) " +
        "VALUES (@IdCompra, @IdMaterial, @Cantidad)";

    SqlCommand cm = new SqlCommand(InsertInfo, nuevaConexion.AbrirConexion());

    cm.Parameters.AddWithValue("@IdCompra", idCompra);

    cm.Parameters.AddWithValue("@IdMaterial", idMaterial);

```

```

        cm.Parameters.AddWithValue("@Cantidad", cantidad);

        cm.ExecuteNonQuery();

        nuevaConexion.CerrarConexion();

        // Notificar que se ha actualizado.
        OnDetalleCompraUpdated();
    }

    /*
    ModificaDato
    Modifica el registro seleccionado en la tabla Comercial.Compra con los datos actualizados.
    */
    public void ModificaDato()
    {
        cantidad = txtCantidad.Text;

        string materialSeleccionado = cmb_Material.Text;

        string[] partesMaterial = materialSeleccionado.Split('-');

        string nombreMaterial = partesMaterial[0].Trim();

        string tipoMaterial = partesMaterial[1].Trim();

        nuevaConexion.AbrirConexion();

        //Rescatar la concatenación del material.

        string queryIdMaterial = "SELECT idMaterial FROM Operativo.Material WHERE NombreMaterial = @Nombre AND TipoMaterial = @Tipo";

        SqlCommand cmdIdMaterial = new SqlCommand(queryIdMaterial, nuevaConexion.AbrirConexion());

        cmdIdMaterial.Parameters.AddWithValue("@Nombre", nombreMaterial);

        cmdIdMaterial.Parameters.AddWithValue("@Tipo", tipoMaterial);

        idMaterial = cmdIdMaterial.ExecuteScalar()?.ToString();

        nuevaConexion.CerrarConexion();

        int cant = int.Parse(cantidad);

        nuevaConexion.AbrirConexion();

        // Cadena SQL para modificar un nuevo registro en la tabla Comercial.DetalleCompra.

        string updateQuery = "UPDATE Comercial.DetalleCompra SET cantidad = @Cantidad WHERE idCompra = @IdCompra AND idMaterial = @IdMaterial";

        SqlCommand cmd = new SqlCommand(updateQuery, nuevaConexion.AbrirConexion());

```

```

cmd.Parameters.AddWithValue("@Cantidad", cantidad);

cmd.Parameters.AddWithValue("@IdCompra", idCompra);

cmd.Parameters.AddWithValue("@IdMaterial", idMaterial);

cmd.ExecuteNonQuery();

nuevaConexion.CerrarConexion();

// Notificar que se ha actualizado.
OnDetalleCompraUpdated();
}

/*
EliminaDato
Elimina el registro seleccionado en la tabla Comercial.Compra con los datos actualizados.
*/
public void EliminaDato()
{
    string materialSeleccionado = cmb_Material.Text;

    string[] partesMaterial = materialSeleccionado.Split('-');

    string nombreMaterial = partesMaterial[0].Trim();

    string tipoMaterial = partesMaterial[1].Trim();

    nuevaConexion.AbrirConexion();

    //Rescatar la concatenación del material.

    string queryIdMaterial = "SELECT idMaterial FROM Operativo.Material WHERE NombreMaterial = @Nombre AND TipoMaterial = @Tipo";

    SqlCommand cmdIdMaterial = new SqlCommand(queryIdMaterial, nuevaConexion.AbrirConexion());

    cmdIdMaterial.Parameters.AddWithValue("@Nombre", nombreMaterial);

    cmdIdMaterial.Parameters.AddWithValue("@Tipo", tipoMaterial);

    idMaterial = cmdIdMaterial.ExecuteScalar()?.ToString();

    nuevaConexion.CerrarConexion();

    nuevaConexion.AbrirConexion();

    // Cadena SQL para eliminar un nuevo registro en la tabla Comercial.DetalleCompra.

    string delete = "DELETE FROM Comercial.DetalleCompra WHERE idCompra = @IdCompra AND idMaterial = @IdMaterial";

    SqlCommand cmd = new SqlCommand(delete, nuevaConexion.AbrirConexion());

```

```

cmd.Parameters.AddWithValue("@IdCompra", idCompra);
cmd.Parameters.AddWithValue("@IdMaterial", idMaterial);
cmd.ExecuteNonQuery();

nuevaConexion.CerrarConexion();

// Notificar que se ha actualizado.
OnDetalleCompraUpdated();
}

/*
    Dispara el Evento para notificar el cambio del subtotal.
*/
protected virtual void OnDetalleCompraUpdated()
{
    SubtotalChanged?.Invoke(this, EventArgs.Empty);
}

/*
    btnInserta_Click

    Evento de clic del botón de inserción.

    Llama a InsertaDato, actualiza el DataGridView llamando a ConsultaDatos y limpia los campos de entrada.
*/
private void btnInserta_Click(object sender, EventArgs e)
{
    InsertaDato();
    ConsultaDatos();
    cmb_Material.SelectedIndex = -1;
    txtCantidad.Clear();
}

/*
    btnModifica_Click

    Evento de clic del botón de modificación.

    Llama a ModificaDato, actualiza el DataGridView llamando a ConsultaDatos y limpia los campos de entrada.
*/
private void btnModifica_Click(object sender, EventArgs e)

```



```

{
    ModificaDato();
    ConsultaDatos();
    cmb_Material.SelectedIndex = -1;
    txtCantidad.Clear();
}

/*
    btnElimina_Click

    Evento de clic del botón de eliminación.

    Llama a EliminaDato, actualiza el DataGridView llamando a ConsultaDatos y limpia los campos de entrada.
*/
private void btnElimina_Click(object sender, EventArgs e)
{
    EliminaDato();
    ConsultaDatos();
    cmb_Material.SelectedIndex = -1;
    txtCantidad.Clear();
}
}
}

```

Formulario Servicio.

Código documentado del Formulario Servicio.

```

// Librerías del sistema y .NET Framework usadas para crear y gestionar el formulario Servicio.
using System;

using System.Collections.Generic;

using System.ComponentModel;

```

```

using System.Data;

using System.Data.SqlClient;

using System.Drawing;

using System.Linq;

using System.Text;

using System.Threading.Tasks;

using System.Windows.Forms;

namespace ServiciosDeMantenimiento
{
    /*
        Clase para gestionar el manejo de los servicio dentro de los Servicios de Mantenimiento
        Proporciona métodos para consultar, insertar, modificar y eliminar datos.
    */

    public partial class Servicio : Form
    {
        // Crea una instancia de una nueva conexión para conectarse con la base de datos de SQL Server.
        ConexionDB nuevaConexion = new ConexionDB("Server=ASUS\\SQLEXPRESS1; DATABASE = ServiciosdeMantenimiento; " +
            "integrated security = true");

        int idServicio = -1;

        string idDomicilio = "";

        string fechaCotizacion = "";

        string fechaInicio = "";

        string fechaFinal = "";

        int duracion = 0;

        decimal costoFinal = 0;

        //Lista que almacena objetos de tipo MaterialXServicio.
        private List<MaterialXServicio> materialXServicioForms = new List<MaterialXServicio>();

        //Lista que almacena objetos de tipo TrabajadorXServicio.
        private List<TrabajadorXServicio> trabajadorXServicioForms = new List<TrabajadorXServicio>();

        /*
            Constructor del formulario Servicio.
            Inicializa el formulario y carga los datos de la tabla Servicio en el DataGridView.
            También suscribe el evento CellClick de la tabla.
        */
    }
}

```

```

*/

public Servicio()
{
    InitializeComponent();

    ConsultaDatos();

    CargarDomicilios();

    TablaServicio.CellClick += TablaServicio_CellClick;
}

/*
ConsultaDatos

Recupera y muestra todos los registros de la tabla Comercial.Compra en el DataGridView.

Formatea la columna de precios y limpia las filas antes de agregar los datos.
*/

public void ConsultaDatos()
{
    nuevaConexion.AbrirConexion();

    TablaServicio.DataSource = null;

    // Cadena SQL para consultar la tabla Operativo.Servicio.

    string selectInfo = "SELECT s.idServicio, s.idDomicilio, s.fechaCotizacion, s.fechaInicio, s.fechaFinal, s.duracion,
s.costoFinal, " +

        "CONCAT(dc.domicilioCliente, '(', c.nombreCliente, ')') AS DomicilioCliente " +

        "FROM Operativo.Servicio s " +

        "JOIN Comercial.DomicilioCliente dc ON s.idDomicilio = dc.idDomicilio " +

        "JOIN Comercial.Cliente c ON dc.idCliente = c.idCliente";

    SqlCommand cmd = new SqlCommand(selectInfo, nuevaConexion.AbrirConexion());

    SqlDataReader reader = cmd.ExecuteReader();

    // Agrega columnas en la TablaServicio si es que no contiene columnas.
    if (TablaServicio.Columns.Count == 0)
    {
        TablaServicio.Columns.Add("idServicio", "ID");

        TablaServicio.Columns.Add("idDomicilio", "Domicilio");

        TablaServicio.Columns.Add("fechaCotizacion", "Fecha Cotizacion");

        TablaServicio.Columns.Add("fechaInicio", "Fecha Inicio");
    }

```

```

        TablaServicio.Columns.Add("fechaFinal", "Fecha Finalización");

        TablaServicio.Columns.Add("duracion", "Duración");

        TablaServicio.Columns.Add("costoFinal", "Costo Final");


        TablaServicio.Columns["costoFinal"].DefaultCellStyle.Format = "N2";
    }


    TablaServicio.Rows.Clear();

    int i = 0;


    // Agrega a TablaServicio una nueva fila para almacenar con los valores correspondientes a las columnas.
    while (reader.Read())
    {
        string domicilioCliente = reader["DomicilioCliente"].ToString();

        int duracion = reader["duracion"] != DBNull.Value ? Convert.ToInt32(reader["duracion"]) + 1 : 1;

        decimal costoFinal = reader["costoFinal"] != DBNull.Value ? Convert.ToDecimal(reader["costoFinal"]) : 0;


        TablaServicio.Rows.Add();

        TablaServicio.Rows[i].Cells[0].Value = reader["idServicio"].ToString();

        TablaServicio.Rows[i].Cells[1].Value = domicilioCliente;

        TablaServicio.Rows[i].Cells[2].Value = Convert.ToDateTime(reader["fechaCotizacion"]).ToString("yyyy-MM-dd");

        TablaServicio.Rows[i].Cells[3].Value = Convert.ToDateTime(reader["fechaInicio"]).ToString("yyyy-MM-dd");

        TablaServicio.Rows[i].Cells[4].Value = Convert.ToDateTime(reader["fechaFinal"]).ToString("yyyy-MM-dd"); ;

        TablaServicio.Rows[i].Cells[5].Value = duracion.ToString();

        TablaServicio.Rows[i].Cells[6].Value = costoFinal.ToString("N2");

        i++;
    }

    reader.Close();

    nuevaConexion.CerrarConexion();
}


/*
    Carga la información de la tabla Comercial.Cliente y Comercial.DomicilioCliente y la muestra en un combobox.
*/

public void CargarDomicilios()
{
    nuevaConexion.AbrirConexion();

```

```

string selectInfo = "SELECT dc.domicilioCliente, c.nombreCliente " +
    "FROM Comercial.DomicilioCliente dc " +
    "JOIN Comercial.Cliente c ON dc.idCliente = c.idCliente";

SqlCommand cmd = new SqlCommand(selectInfo, nuevaConexion.AbrirConexion());

SqlDataReader reader = cmd.ExecuteReader();

comboBoxDomicilio.Items.Clear();

// Lee cada domicilio y cliente del resultado de la consulta
while (reader.Read())
{
    string domicilioCliente = reader["domicilioCliente"].ToString();
    string nombreCliente = reader["nombreCliente"].ToString();
    string domicilio = $"{domicilioCliente} ({nombreCliente})";

    // Agrega el resultado al combo box
    comboBoxDomicilio.Items.Add(domicilio);
}

reader.Close();
nuevaConexion.CerrarConexion();
}

/*
TablaServicio_CellClick

Evento que se activa al hacer clic en una celda del DataGridView.

Llena los campos de entrada con los datos de la fila seleccionada.
*/

public void TablaServicio_CellClick(object sender, DataGridViewCellEventArgs e)
{
    if (e.RowIndex >= 0)
    {
        DataGridViewRow row = TablaServicio.Rows[e.RowIndex];

        idServicio = int.Parse(row.Cells[0].Value.ToString());
        comboBoxDomicilio.Text = row.Cells[1].Value.ToString();
        dtpFech_Cot.Text = row.Cells[2].Value.ToString();
        dtpFech_Ini.Text = row.Cells[3].Value.ToString();
    }
}

```

```

        dtpFech_Fin.Text = row.Cells[4].Value.ToString();

        // Hace visible al botón btnMaterial.
        btnMaterial.Visible = true;

        // Hace visible al botón btnTrabajador.
        btnTrabajador.Visible = true;
    }
}

/*
InsertaDato
Inserta un nuevo registro en la tabla Operativo.Servicio con los datos ingresados.
*/
public void InsertaDato()
{
    string domicilioSeleccionado = cmbBoxDomicilio.Text;
    string[] partesDomicilio = domicilioSeleccionado.Split(';');
    string domicilioCliente = partesDomicilio[0].Trim();
    string fechaCotizacion = dtpFech_Cot.Value.ToString("yyyy-MM-dd");
    string fechaInicio = dtpFech_Ini.Value.ToString("yyyy-MM-dd");
    string fechaFinal = dtpFech_Fin.Value.ToString("yyyy-MM-dd");

    // Validación de fechas
    if (dtpFech_Ini.Value < dtpFech_Cot.Value)
    {
        MessageBox.Show("La fecha de inicio no puede ser anterior a la fecha de cotización.", "Error de Fecha",
        MessageBoxButtons.OK, MessageBoxIcon.Error);

        return;
    }

    if (dtpFech_Fin.Value < dtpFech_Ini.Value)
    {
        MessageBox.Show("La fecha de finalización no puede ser anterior a la fecha de inicio.", "Error de Fecha",
        MessageBoxButtons.OK, MessageBoxIcon.Error);

        return;
    }
}

```

```

string idDomicilio = "";

nuevaConexion.AbrirConexion();

//Rescatar la concatenación del domicilio.

string queryIdDomicilio = "SELECT idDomicilio FROM Comercial.DomicilioCliente WHERE domicilioCliente = @Domicilio";

using (SqlCommand cmdIdDomicilio = new SqlCommand(queryIdDomicilio, nuevaConexion.AbrirConexion()))
{
    cmdIdDomicilio.Parameters.AddWithValue("@Domicilio", domicilioCliente);
    var result = cmdIdDomicilio.ExecuteScalar();

    if (result != null)
        idDomicilio = result.ToString();
}

nuevaConexion.CerrarConexion();

// Verificar si ya existe un servicio programado para este domicilio antes de la fecha de inicio seleccionada
string queryCheckServicio = "SELECT COUNT(*) FROM Operativo.Servicio " +
    "WHERE idDomicilio = @IdDomicilio AND fechaFinal >= @FechaInicio";

using (SqlCommand cmdCheckServicio = new SqlCommand(queryCheckServicio, nuevaConexion.AbrirConexion()))
{
    cmdCheckServicio.Parameters.AddWithValue("@IdDomicilio", idDomicilio);
    cmdCheckServicio.Parameters.AddWithValue("@FechaInicio", fechaInicio);
    int count = (int)cmdCheckServicio.ExecuteScalar();

    if (count > 0)
    {
        MessageBox.Show("No se puede programar un nuevo servicio para este domicilio hasta que el servicio actual finalice.");
        nuevaConexion.CerrarConexion();
        return;
    }
}

nuevaConexion.AbrirConexion();

// Cadena SQL para agregar un nuevo registro en la tabla Operativo.Servicio.

```

```

string insertInfo = "INSERT INTO Operativo.Servicio(idDomicilio, fechaCotizacion, fechaInicio, fechaFinal) " +
    "VALUES (@IdDomicilio, @FechaCot, @FechaIni, @FechaFin)";

using (SqlCommand cm = new SqlCommand(insertInfo, nuevaConexion.AbrirConexion()))
{
    cm.Parameters.AddWithValue("@IdDomicilio", idDomicilio);
    cm.Parameters.AddWithValue("@FechaCot", fechaCotizacion);
    cm.Parameters.AddWithValue("@FechaIni", fechaInicio);
    cm.Parameters.AddWithValue("@FechaFin", fechaFinal);
    cm.ExecuteNonQuery();
}

nuevaConexion.CerrarConexion();
}

/*
ModificaDato
Modifica el registro seleccionado en la tabla Operativo.Servicio con los datos actualizados.
*/

public void ModificaDato()
{
    string domicilioSeleccionado = cmbBoxDomicilio.Text;
    string idDomicilio = "";
    string fechaCotizacion = dtpFech_Cot.Value.ToString("yyyy-MM-dd");
    string fechaInicio = dtpFech_Ini.Value.ToString("yyyy-MM-dd");
    string fechaFinal = dtpFech_Fin.Value.ToString("yyyy-MM-dd");

    // Validación de fechas
    if (dtpFech_Ini.Value < dtpFech_Cot.Value)
    {
        MessageBox.Show("La fecha de inicio no puede ser anterior a la fecha de cotización.", "Error de Fecha",
            MessageBoxButtons.OK, MessageBoxIcon.Error);

        return;
    }

    if (dtpFech_Fin.Value < dtpFech_Ini.Value)
    {
        MessageBox.Show("La fecha de finalización no puede ser anterior a la fecha de inicio.", "Error de Fecha",
            MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}

```



```

        return;
    }

    nuevaConexion.AbrirConexion();

    //Rescatar la concatenación del domicilio.
    string queryIdDomicilio = "SELECT dc.idDomicilio FROM Comercial.DomicilioCliente dc " +
        "JOIN Comercial.Cliente c ON dc.idCliente = c.idCliente " +
        "WHERE CONCAT(dc.domicilioCliente, '(', c.nombreCliente, ')') = @Domicilio";

    SqlCommand cmdIdDomicilio = new SqlCommand(queryIdDomicilio, nuevaConexion.AbrirConexion());
    cmdIdDomicilio.Parameters.AddWithValue("@Domicilio", domicilioSeleccionado);
    var result = cmdIdDomicilio.ExecuteScalar();
    if (result != null)
    {
        idDomicilio = result.ToString();
    }
    nuevaConexion.CerrarConexion();

    // Verificar si ya existe un servicio programado para este domicilio antes de la fecha de inicio seleccionada
    string queryCheckServicio = "SELECT COUNT(*) FROM Operativo.Servicio " +
        "WHERE idDomicilio = @IdDomicilio AND fechaFinal >= @FechaInicio";
    using (SqlCommand cmdCheckServicio = new SqlCommand(queryCheckServicio, nuevaConexion.AbrirConexion()))
    {
        cmdCheckServicio.Parameters.AddWithValue("@IdDomicilio", idDomicilio);
        cmdCheckServicio.Parameters.AddWithValue("@FechaInicio", fechaInicio);
        int count = (int)cmdCheckServicio.ExecuteScalar();

        if (count > 0)
        {
            MessageBox.Show("No se puede programar un nuevo servicio para este domicilio hasta que el servicio actual finalice.");
            nuevaConexion.CerrarConexion();
            return;
        }
    }

    nuevaConexion.AbrirConexion();

```

```

// Cadena SQL para agregar un nuevo registro en la tabla Operativo.Servicio.

string update = "UPDATE Operativo.Servicio SET idDomicilio = @IdDomicilio, fechaCotizacion = @FechaCot, " +
    "fechaInicio = @FechaIni, fechaFinal = @FechaFin WHERE idServicio = @IdServicio";

SqlCommand cmd = new SqlCommand(update, nuevaConexion.AbrirConexion());

cmd.Parameters.AddWithValue("@IdDomicilio", idDomicilio);

cmd.Parameters.AddWithValue("@FechaCot", fechaCotizacion);

cmd.Parameters.AddWithValue("@FechaIni", fechaInicio);

cmd.Parameters.AddWithValue("@FechaFin", fechaFinal);

cmd.Parameters.AddWithValue("@IdServicio", idServicio);

cmd.ExecuteNonQuery();

nuevaConexion.CerrarConexion();
}

/*
EliminaDato

Elimina el registro seleccionado en la tabla Operativo.Servicio con los datos actualizados.
*/

public void EliminaDato()
{
    if (idServicio != -1)
    {
        nuevaConexion.AbrirConexion();

        // Eliminar primero los materiales por servicio.

        string deleteMaterialXServicio = "DELETE FROM Operativo.MaterialXServicio WHERE idServicio = @Id";

        SqlCommand cmdMaterialXServicio = new SqlCommand(deleteMaterialXServicio, nuevaConexion.AbrirConexion());

        cmdMaterialXServicio.Parameters.AddWithValue("@Id", idServicio);

        cmdMaterialXServicio.ExecuteNonQuery();

        // Eliminar primero los trabajadores por servicio.

        string deleteMaterialXTrabajador = "DELETE FROM Operativo.TrabajadorXServicio WHERE idServicio = @Id";

        SqlCommand cmdTrabajadorXServicio = new SqlCommand(deleteMaterialXTrabajador, nuevaConexion.AbrirConexion());

        cmdTrabajadorXServicio.Parameters.AddWithValue("@Id", idServicio);

        cmdTrabajadorXServicio.ExecuteNonQuery();

        // Cadena SQL para eliminar un nuevo registro en la tabla Comercial.Compra.

```

```

string delete = "DELETE FROM Operativo.Servicio WHERE idServicio = @Id";

SqlCommand cmd = new SqlCommand(delete, nuevaConexion.AbrirConexion());

cmd.Parameters.AddWithValue("@Id", idServicio);

cmd.ExecuteNonQuery();


string checkEmptyQuery = "SELECT COUNT(*) FROM Operativo.Servicio";

SqlCommand checkCmd = new SqlCommand(checkEmptyQuery, nuevaConexion.AbrirConexion());

int rowCount = (int)checkCmd.ExecuteScalar();


// Reinicia el valor del idServicio cuando la cantidad de filas es igual a 0.
if (rowCount == 0)
{
    string resetIdentityQuery = "DBCC CHECKIDENT ('Operativo.Servicio', RESEED, 0)";

    SqlCommand resetCmd = new SqlCommand(resetIdentityQuery, nuevaConexion.AbrirConexion());

    resetCmd.ExecuteNonQuery();

}

nuevaConexion.CerrarConexion();

}

}

/*

btnInserta_Click

Evento de clic del botón de inserción.

Llama a InsertaDato, actualiza el DataGridView llamando a ConsultaDatos y limpia los campos de entrada.

*/

private void btnInserta_Click(object sender, EventArgs e)
{
    InsertaDato();

    ConsultaDatos();

    cmbBoxDomicilio.SelectedIndex = -1;

}

/*

btnModifica_Click

Evento de clic del botón de modificación.

Llama a ModificaDato, actualiza el DataGridView llamando a ConsultaDatos y limpia los campos de entrada.

*/

```

```

private void btnModifica_Click(object sender, EventArgs e)
{
    ModificaDato();
    ConsultaDatos();
    cmbBoxDomicilio.SelectedIndex = -1;
}

/*
    btnElimina_Click
    Evento de clic del botón de eliminación.
    Llama a EliminaDato, actualiza el DataGridView llamando a ConsultaDatos y limpia los campos de entrada.
*/
private void btnElimina_Click(object sender, EventArgs e)
{
    EliminaDato();
    ConsultaDatos();
    cmbBoxDomicilio.SelectedIndex = -1;
}

/*
    Evento que se dispara al hacer clic en el botón de Taterial.
    Abre el formulario DetalleCompra para gestionar los materiales asociados a un servicio específico.
*/
private void btnMaterial_Click(object sender, EventArgs e)
{
    if (idServicio != -1)
    {
        MaterialXServicio materialXServicioForm = new MaterialXServicio(idServicio);
        materialXServicioForms.Add(materialXServicioForm);
        materialXServicioForm.SubtotalChanged += Form_SubtotalChanged;
        materialXServicioForm.FormClosed += (s, args) =>
        {
            materialXServicioForms.Remove(materialXServicioForm);
        };
        materialXServicioForm.Show();
    }
}

```

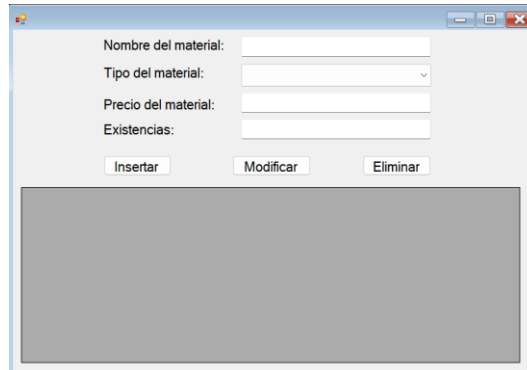
```

/*
    Evento que se dispara al hacer clic en el botón de Trabajador.
    Abre el formulario DetalleCompra para gestionar los trabajadores asociados a un servicio específico.
*/
private void btnTrabajador_Click(object sender, EventArgs e)
{
    if (idServicio != -1)
    {
        TrabajadorXServicio trabajadorXServicioForm = new TrabajadorXServicio(idServicio);
        trabajadorXServicioForms.Add(trabajadorXServicioForm);
        trabajadorXServicioForm.SubtotalChanged += Form_SubtotalChanged;
        trabajadorXServicioForm.FormClosed += (s, args) =>
        {
            trabajadorXServicioForms.Remove(trabajadorXServicioForm);
        };
        trabajadorXServicioForm.Show();
    }
}

/*
    Ejecuta el Evento el cual es disparado por una instancia de DetalleCompra.
*/
private void Form_SubtotalChanged(object sender, EventArgs e)
{
    ConsultaDatos();
}
}

```

Formulario Material.



Código documentado del Formulario Material.

// Librerías del sistema y .NET Framework usadas para crear y gestionar el formulario Material.

```
using System;
```

```
using System.Collections.Generic;
```

```
using System.ComponentModel;
```

```
using System.Data;
```

```
using System.Data.SqlClient;
```

```
using System.Drawing;
```

```
using System.Linq;
```

```
using System.Text;
```

```
using System.Threading.Tasks;
```

```
using System.Windows.Forms;
```

```
namespace ServiciosDeMantenimiento
```

```
{
```

```
    /*
```

```
        Clase para gestionar el manejo de los materiales dentro de los Servicios de Mantenimiento
```

```
        Proporciona métodos para consultar, insertar, modificar y eliminar datos.
```

```
    */
```

```
    public partial class Material : Form
```

```
    {
```

```
        // Crea una instancia de una nueva conexión para conectarse con la base de datos de SQL Server.
```

```
        ConexionDB nuevaConexion = new ConexionDB("Server=ASUS\\SQLEXPRESS1; DATABASE = ServiciosdeMantenimiento; " +
```

```
            "integrated security = true");
```

```
        int idMaterial = -1;
```

```
        string nombreMaterial = "";
```

```

string tipoMaterial = "";

string precioMaterial = "";

string existencia = "";

/*
    Constructor del formulario Material.

    Inicializa el formulario y carga los datos de la tabla Material en el DataGridView.

    También suscribe el evento CellClick de la tabla.
*/

public Material()
{
    InitializeComponent();

    ConsultaDatos();

    TablaMaterial.CellClick += TablaMaterial_CellClick;
}

/*
    ConsultaDatos

    Recupera y muestra todos los registros de la tabla Operativo.Material en el DataGridView.

    Formatea las columnas y limpia las filas antes de agregar los datos.
*/

public void ConsultaDatos()
{
    nuevaConexion.AbrirConexion();

    TablaMaterial.DataSource = null;

    // Cadena SQL para consultar la tabla Operativo.Material.
    string selectInfo = "SELECT * FROM Operativo.Material";

    SqlCommand cmd = new SqlCommand(selectInfo, nuevaConexion.AbrirConexion());

    SqlDataReader reader = cmd.ExecuteReader();

    // Agrega columnas en la TablaMaterial si es que no contiene columnas.
    if (TablaMaterial.Columns.Count == 0)
    {
        TablaMaterial.Columns.Add("idMaterial", "ID");

        TablaMaterial.Columns.Add("nombreMaterial", "Nombre del material");

        TablaMaterial.Columns.Add("tipoMaterial", "Tipo del material");
    }
}

```

```

        TablaMaterial.Columns.Add("precioMaterial", "Precio del material");

        TablaMaterial.Columns.Add("existencia", "Existencias");


        TablaMaterial.Columns["precioMaterial"].DefaultCellStyle.Format = "N2";
    }

    TablaMaterial.Rows.Clear();

    int i = 0;

    // Agrega a TablaMaterial una nueva fila para almacenar con los valores correspondientes a las columnas.
    while (reader.Read())
    {
        TablaMaterial.Rows.Add();

        TablaMaterial.Rows[i].Cells[0].Value = reader["idMaterial"].ToString();

        TablaMaterial.Rows[i].Cells[1].Value = reader["nombreMaterial"].ToString();

        TablaMaterial.Rows[i].Cells[2].Value = reader["tipoMaterial"].ToString();

        TablaMaterial.Rows[i].Cells[3].Value = Convert.ToDecimal(reader["precioMaterial"]).ToString("N2");

        TablaMaterial.Rows[i].Cells[4].Value = reader["existencia"].ToString();

        i++;
    }

    reader.Close();

    nuevaConexion.CerrarConexion();
}

/*
    TablaMaterial_CellClick

    Evento que se activa al hacer clic en una celda del DataGridView.

    Llena los campos de entrada con los datos de la fila seleccionada.
*/

private void TablaMaterial_CellClick(object sender, DataGridViewCellEventArgs e)
{
    if (e.RowIndex >= 0)
    {
        DataGridViewRow row = TablaMaterial.Rows[e.RowIndex];

        idMaterial = int.Parse(row.Cells[0].Value.ToString());

        txtNombre_Material.Text = row.Cells[1].Value.ToString();

        cmbBoxTipo_Material.Text = row.Cells[2].Value.ToString();
    }
}

```



```

        txtPrecio_Material.Text = row.Cells[3].Value.ToString();

        txtExistencia.Text = row.Cells[4].Value.ToString();

    }

}

/*
InsertaDato

Inserta un nuevo registro en la tabla Operativo.Material con los datos ingresados.

Si ocurre un error (como duplicación de nombre), muestra un mensaje de error.
*/

public void InsertaDato()
{
    nombreMaterial = txtNombre_Material.Text;
    tipoMaterial = cmbBoxTipo_Material.Text;
    precioMaterial = txtPrecio_Material.Text;
    existencia = txtExistencia.Text;

    try
    {
        nuevaConexion.AbrirConexion();

        // Cadena SQL para agregar un nuevo registro en la tabla Operativo.Material.
        string InsertInfo = "INSERT INTO Operativo.Material(nombreMaterial, tipoMaterial, precioMaterial, existencia) " +
            "VALUES ('" + nombreMaterial + "', '" + tipoMaterial + "', '" + precioMaterial + "', '" + existencia + "')";

        SqlCommand cm = new SqlCommand(InsertInfo, nuevaConexion.AbrirConexion());

        cm.ExecuteNonQuery();

    }

    //Verifica si el nombre del material ya existe dentro de Operativo.Material.
    catch (SqlException ex)
    {
        if (ex.Number == 2627)
        {
            MessageBox.Show("No se puede insertar el material. El nombre ya existe.");
        }
        else
        {

```

```

        MessageBox.Show("Error al insertar el material: " + ex.Message);
    }
}

nuevaConexion.CerrarConexion();
}

/*
ModificaDato
Modifica el registro seleccionado en la tabla Operativo.Material con los datos actualizados.
Si ocurre un error (como duplicación de nombre), muestra un mensaje de error
*/
public void ModificaDato()
{
    nombreMaterial = txtNombre_Material.Text;
    tipoMaterial = cmbBoxTipo_Material.Text;
    precioMaterial = txtPrecio_Material.Text;
    existencia = txtExistencia.Text;

    try
    {
        nuevaConexion.AbrirConexion();

        // Cadena SQL para modificar un nuevo registro en la tabla Operativo.Material.
        string update = "UPDATE Operativo.Material SET nombreMaterial = @Nombre, tipoMaterial = @Tipo, precioMaterial = @Precio, existencia = @Existencias " +
            "WHERE idMaterial = @Id";

        SqlCommand cmd = new SqlCommand(update, nuevaConexion.AbrirConexion());
        cmd.Parameters.AddWithValue("@Nombre", nombreMaterial);
        cmd.Parameters.AddWithValue("@Tipo", tipoMaterial);
        cmd.Parameters.AddWithValue("@Precio", precioMaterial);
        cmd.Parameters.AddWithValue("@Existencias", existencia);
        cmd.Parameters.AddWithValue("@Id", idMaterial);
        cmd.ExecuteNonQuery();
    }

    //Verifica si el nombre del material ya existe dentro de Operativo.Material.
    catch (SqlException ex)

```

```

{
    if (ex.Number == 2627)
    {
        MessageBox.Show("No se puede insertar el material. El nombre ya existe.");
    }
    else
    {
        MessageBox.Show("Error al insertar el material: " + ex.Message);
    }
}

nuevaConexion.CerrarConexion();
}

/*
EliminaDato
Elimina el registro seleccionado de la tabla Operativo.Material.
Si la tabla queda vacía, reinicia el contador de identidad.
*/

public void EliminaDato()
{
    if (idMaterial != -1)
    {
        nuevaConexion.AbrirConexion();

        // Cadena SQL para eliminar un nuevo registro en la tabla Operativo.Material .
        string delete = "DELETE FROM Operativo.Material WHERE idMaterial = @Id";
        SqlCommand cmd = new SqlCommand(delete, nuevaConexion.AbrirConexion());
        cmd.Parameters.AddWithValue("@Id", idMaterial);
        cmd.ExecuteNonQuery();

        string checkEmptyQuery = "SELECT COUNT(*) FROM Operativo.Material";
        SqlCommand checkCmd = new SqlCommand(checkEmptyQuery, nuevaConexion.AbrirConexion());
        int rowCount = (int)checkCmd.ExecuteScalar();

        // Reinicia el valor del idMaterial cuando la cantidad de filas es igual a 0.
        if (rowCount == 0)
        {

```

```

        string resetIdentityQuery = "DBCC CHECKIDENT ('Operativo.Material', RESEED, 0)";

        SqlCommand resetCmd = new SqlCommand(resetIdentityQuery, nuevaConexion.AbrirConexion());

        resetCmd.ExecuteNonQuery();

    }

    nuevaConexion.CerrarConexion();

}

}

/*

    btnInserta_Click

    Evento de clic del botón de inserción.

    Llama a InsertaDato, actualiza el DataGridView llamando a ConsultarDatos y limpia los campos de entrada.

*/

private void btnInserta_Click(object sender, EventArgs e)
{
    InsertaDato();

    ConsultarDatos();

    txtNombre_Material.Clear();

    cmbBoxTipo_Material.SelectedIndex = -1;

    txtPrecio_Material.Clear();

    txtExistencia.Clear();

    idMaterial = -1;
}

/*

    btnModifica_Click

    Evento de clic del botón de modificación.

    Llama a ModificaDato, actualiza el DataGridView llamando a ConsultarDatos y limpia los campos de entrada.

*/

private void btnModifica_Click(object sender, EventArgs e)
{
    ModificaDato();

    ConsultarDatos();

    txtNombre_Material.Clear();

    cmbBoxTipo_Material.SelectedIndex = -1;

    txtPrecio_Material.Clear();

    txtExistencia.Clear();
}

```

```

        idMaterial = -1;
    }

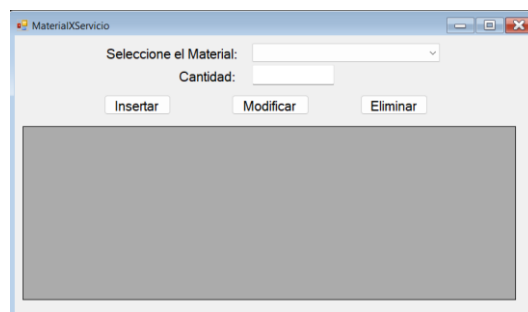
    /*
    btnElimina_Click

    Evento de clic del botón de eliminación.

    Llama a EliminaDato, actualiza el DataGridView llamando a ConsultaDatos y limpia los campos de entrada.
    */
    private void btnElimina_Click(object sender, EventArgs e)
    {
        EliminaDato();
        ConsultaDatos();
        txtNombre_Material.Clear();
        cmbBoxTipo_Material.SelectedIndex = -1;
        txtPrecio_Material.Clear();
        txtExistencia.Clear();
        idMaterial = -1;
    }
}
}
}

```

Formulario MaterialXServicio.



Código documentado del Formulario MaterialXServicio.

// Librerías del sistema y .NET Framework usadas para crear y gestionar el formulario MarerialXServicio.

```

using System;

using System.Collections.Generic;

using System.ComponentModel;

using System.Data;

using System.Data.SqlClient;

using System.Drawing;

```

```

using System.Linq;

using System.Text;

using System.Threading.Tasks;

using System.Windows.Forms;

namespace ServiciosDeMantenimiento
{
    /*
        Clase para gestionar el manejo de los materiales de los servicios dentro de los Servicios de Mantenimiento
        Proporciona métodos para consultar, insertar, modificar y eliminar datos.
    */

    public partial class MaterialXServicio : Form
    {
        // Crea una instancia de una nueva conexión para conectarse con la base de datos de SQL Server.
        ConexionDB nuevaConexion = new ConexionDB("Server=ASUS\\SQLEXPRESS1; DATABASE = ServiciosdeMantenimiento; " +
            "integrated security = true");

        int idServicio;

        string idMaterial = "";

        int material;

        string cantidad;

        float subtotalMaterial = 0;

        // Define un evento llamado SubtotalChanged el cual se utiliza para notificar cuando ocurre un cambio en el subtotal de un
        servicio.

        public event EventHandler SubtotalChanged;

        /*
            Constructor del formulario MaterialXServicio.
            Inicializa el formulario y carga los datos de la tabla MaterialXServicio en el DataGridView.
            También suscribe el evento CellClick de la tabla.
        */

        public MaterialXServicio(int idServicio)
        {
            InitializeComponent();

            this.idServicio = idServicio;

            ConsultaDatos();

            CargarMateriales();
        }
    }
}

```

```

        TablaMaterialXServicio.CellClick += TablaMaterialXServicio_CellClick;
    }

    /*
    ConsultaDatos
    Recupera y muestra todos los registros de la tabla Operativo.MaterialXServicio en el DataGridView.
    Formatea la columna de precios y limpia las filas antes de agregar los datos.
    */
    public void ConsultaDatos()
    {
        nuevaConexion.AbrirConexion();
        TablaMaterialXServicio.DataSource = null;

        // Cadena SQL para consultar la tabla Operativo.MaterialXServicio.
        string selectInfo = "SELECT s.idServicio, CONCAT(s.idServicio, ' - ', c.nombreCliente) AS ServicioCliente, " +
            "mat.NombreMaterial, mat.TipoMaterial, m.cantMaterial, m.subtotalMaterial " +
            "FROM Operativo.MaterialXServicio m " +
            "JOIN Operativo.Servicio s ON m.idServicio = s.idServicio " +
            "JOIN Comercial.DomicilioCliente dc ON s.idDomicilio = dc.idDomicilio " +
            "JOIN Comercial.Cliente c ON dc.idCliente = c.idCliente " +
            "JOIN Operativo.Material mat ON m.idMaterial = mat.idMaterial " +
            "WHERE m.idServicio = @IdServicio";

        SqlCommand cmd = new SqlCommand(selectInfo, nuevaConexion.AbrirConexion());
        cmd.Parameters.AddWithValue("@IdServicio", idServicio);
        SqlDataReader reader = cmd.ExecuteReader();

        // Agrega columnas en la TablaMaterialXServicio si es que no contiene columnas.
        if (TablaMaterialXServicio.Columns.Count == 0)
        {
            TablaMaterialXServicio.Columns.Add("ServicioCliente", "Servicio");
            TablaMaterialXServicio.Columns.Add("idMaterial", "Material");
            TablaMaterialXServicio.Columns.Add("cantMaterial", "Cantidad");
            TablaMaterialXServicio.Columns.Add("subtotalMaterial", "Subtotal");

            TablaMaterialXServicio.Columns["subtotalMaterial"].DefaultCellStyle.Format = "N2";
        }
    }

```

```

TablaMaterialXServicio.Rows.Clear();

int i = 0;

// Agrega a TablaMaterialXServicio una nueva fila para almacenar con los valores correspondientes a las columnas.
while (reader.Read())
{
    string servicioCliente = reader["ServicioCliente"].ToString();
    string nombreMaterial = reader["NombreMaterial"].ToString();
    string tipoMaterial = reader["TipoMaterial"].ToString();
    string material = $"{nombreMaterial} - {tipoMaterial}";
    decimal subtotalMaterial = reader["subtotalMaterial"] != DBNull.Value ? Convert.ToDecimal(reader["subtotalMaterial"]) : 0;

    TablaMaterialXServicio.Rows.Add();
    TablaMaterialXServicio.Rows[i].Cells[0].Value = servicioCliente;
    TablaMaterialXServicio.Rows[i].Cells[1].Value = material;
    TablaMaterialXServicio.Rows[i].Cells[2].Value = Convert.ToInt32(reader["cantMaterial"]).ToString();
    TablaMaterialXServicio.Rows[i].Cells[3].Value = subtotalMaterial.ToString("N2");
    i++;
}

reader.Close();
nuevaConexion.CerrarConexion();
}

/*
    Carga la información de la tabla Operativo.Material y la muestra en un combobox.
*/
public void CargarMateriales()
{
    nuevaConexion.AbrirConexion();

    string selectInfo = "SELECT NombreMaterial, TipoMaterial FROM Operativo.Material";

    SqlCommand cmd = new SqlCommand(selectInfo, nuevaConexion.AbrirConexion());
    SqlDataReader reader = cmd.ExecuteReader();

    cmb_Material.Items.Clear();

```



```

// Lee cada material del resultado de la consulta
while (reader.Read())
{
    string nombreMaterial = reader["NombreMaterial"].ToString();
    string tipoMaterial = reader["TipoMaterial"].ToString();
    string material = $"{nombreMaterial} - {tipoMaterial}";

    // Agrega el resultado al combo box
    cmb_Material.Items.Add(material);
}

reader.Close();
nuevaConexion.CerrarConexion();
}

/*
TablaMaterialXServicio_CellClick

Evento que se activa al hacer clic en una celda del DataGridView.

Llena los campos de entrada con los datos de la fila seleccionada.
*/
private void TablaMaterialXServicio_CellClick(object sender, DataGridViewCellEventArgs e)
{
    if (e.RowIndex >= 0)
    {
        DataGridViewRow row = TablaMaterialXServicio.Rows[e.RowIndex];
        cmb_Material.Text = row.Cells[1].Value.ToString();
        txtCantidad.Text = row.Cells[2].Value.ToString();
    }
}

/*
InsertaDato

Inserta un nuevo registro en la tabla Operativo.MaterialXServicio con los datos ingresados.
*/
public void InsertaDato()
{

```

```

cantidad = txtCantidad.Text;

string materialSeleccionado = cmb_Material.Text;

string[] partesMaterial = materialSeleccionado.Split('-');

string nombreMaterial = partesMaterial[0].Trim();

string tipoMaterial = partesMaterial[1].Trim();


nuevaConexion.AbrirConexion();


//Rescatar la concatenación del material.

string queryIdMaterial = "SELECT idMaterial, existencia FROM Operativo.Material WHERE NombreMaterial = @Nombre AND
TipoMaterial = @Tipo";

SqlCommand cmdIdMaterial = new SqlCommand(queryIdMaterial, nuevaConexion.AbrirConexion());

cmdIdMaterial.Parameters.AddWithValue("@Nombre", nombreMaterial);

cmdIdMaterial.Parameters.AddWithValue("@Tipo", tipoMaterial);

SqlDataReader reader = cmdIdMaterial.ExecuteReader();


if (reader.Read())
{
    idMaterial = reader["idMaterial"].ToString();

    int existencia = Convert.ToInt32(reader["existencia"]);

    reader.Close();

    nuevaConexion.CerrarConexion();


    int cant = int.Parse(cantidad);


    //Verifica la existencia del material para el servicio.

    if (cant > existencia)
    {
        MessageBox.Show("No hay suficiente existencia del material seleccionado.");

        return;
    }


    nuevaConexion.AbrirConexion();


    // Verificar si ya existe un material para este servicio.

    string verificarMaterial = "SELECT COUNT(*) FROM Operativo.MaterialXServicio WHERE idServicio = @IdServicio AND
idMaterial = @IdMaterial";

```

```

SqlCommand cmdVverificarMaterial = new SqlCommand(verificarMaterial, nuevaConexion.AbrirConexion());

cmdVverificarMaterial.Parameters.AddWithValue("@IdServicio", idServicio);
cmdVverificarMaterial.Parameters.AddWithValue("@IdMaterial", idMaterial);

int count = (int)cmdVverificarMaterial.ExecuteScalar();

if (count > 0)
{
    MessageBox.Show("El material ya está asociado a este servicio.");
    return;
}

nuevaConexion.AbrirConexion();

// Cadena SQL para agregar un nuevo registro en la tabla Operativo.MaterialXServicio.
string InsertInfo = "INSERT INTO Operativo.MaterialXServicio(idServicio, idMaterial, cantMaterial) " +
    "VALUES (@IdServicio, @IdMaterial, @Cantidad)";

SqlCommand cm = new SqlCommand(InsertInfo, nuevaConexion.AbrirConexion());
cm.Parameters.AddWithValue("@IdServicio", idServicio);
cm.Parameters.AddWithValue("@IdMaterial", idMaterial);
cm.Parameters.AddWithValue("@Cantidad", cantidad);
cm.ExecuteNonQuery();
nuevaConexion.CerrarConexion();

// Notificar que se ha actualizado.
OnMaterialXServicioUpdated();
}
}

/*
ModificaDato
Modifica el registro seleccionado en la tabla Operativo.MaterialXServicio con los datos actualizados.
*/
public void ModificaDato()
{
    cantidad = txtCantidad.Text;

```

```

string materialSeleccionado = cmb_Material.Text;

string[] partesMaterial = materialSeleccionado.Split('-');

string nombreMaterial = partesMaterial[0].Trim();

string tipoMaterial = partesMaterial[1].Trim();


nuevaConexion.AbrirConexion();


//Rescatar la concatenación del material.

string queryIdMaterial = "SELECT idMaterial, existencia FROM Operativo.Material WHERE NombreMaterial = @Nombre AND
TipoMaterial = @Tipo";

SqlCommand cmdIdMaterial = new SqlCommand(queryIdMaterial, nuevaConexion.AbrirConexion());

cmdIdMaterial.Parameters.AddWithValue("@Nombre", nombreMaterial);

cmdIdMaterial.Parameters.AddWithValue("@Tipo", tipoMaterial);

SqlDataReader reader = cmdIdMaterial.ExecuteReader();

if (reader.Read())
{
    idMaterial = reader["idMaterial"].ToString();

    int existencia = Convert.ToInt32(reader["existencia"]);

    reader.Close();

    nuevaConexion.CerrarConexion();

    int cant = int.Parse(cantidad);

    nuevaConexion.AbrirConexion();

    //Verifica la existencia del material para el servicio.

    if (cant > existencia)
    {
        MessageBox.Show("No hay suficiente existencia del material seleccionado.");

        return;
    }

    string verificarMaterial = "SELECT COUNT(*) FROM Operativo.MaterialXServicio WHERE idServicio = @IdServicio AND
idMaterial = @IdMaterial";

    SqlCommand cmdVerificarMaterial = new SqlCommand(verificarMaterial, nuevaConexion.AbrirConexion());

    cmdVerificarMaterial.Parameters.AddWithValue("@IdServicio", idServicio);

    cmdVerificarMaterial.Parameters.AddWithValue("@IdMaterial", idMaterial);

```

```

int count = (int)cmdVerificarMaterial.ExecuteScalar();

nuevaConexion.CerrarConexion();

if (count == 0)
{
    MessageBox.Show("El material no está asociado a este servicio. No se puede modificar.");
    return;
}

nuevaConexion.AbrirConexion();

// Cadena SQL para modificar un nuevo registro en la tabla Operativo.MaterialXServicio.
string updateQuery = "UPDATE Operativo.MaterialXServicio SET cantMaterial = @Cantidad WHERE idServicio = @IdServicio AND idMaterial = @IdMaterial";

SqlCommand cmdUpdate = new SqlCommand(updateQuery, nuevaConexion.AbrirConexion());
cmdUpdate.Parameters.AddWithValue("@Cantidad", cant);
cmdUpdate.Parameters.AddWithValue("@IdServicio", idServicio);
cmdUpdate.Parameters.AddWithValue("@IdMaterial", idMaterial);
cmdUpdate.ExecuteNonQuery();
nuevaConexion.CerrarConexion();

// Notificar que se ha actualizado.
OnMaterialXServicioUpdated();
}
}

/*
EliminaDato
Elimina el registro seleccionado en la tabla Operativo.MaterialXServicio con los datos actualizados.
*/
public void EliminaDato()
{
    string materialSeleccionado = cmb_Material.Text;
    string[] partesMaterial = materialSeleccionado.Split('-');
    string nombreMaterial = partesMaterial[0].Trim();
    string tipoMaterial = partesMaterial[1].Trim();

```

```

nuevaConexion.AbrirConexion();

//Rescatar la concatenación del material.

string queryIdMaterial = "SELECT idMaterial FROM Operativo.Material WHERE NombreMaterial = @Nombre AND
TipoMaterial = @Tipo";

SqlCommand cmdIdMaterial = new SqlCommand(queryIdMaterial, nuevaConexion.AbrirConexion());

cmdIdMaterial.Parameters.AddWithValue("@Nombre", nombreMaterial);

cmdIdMaterial.Parameters.AddWithValue("@Tipo", tipoMaterial);

idMaterial = cmdIdMaterial.ExecuteScalar()?.ToString();

nuevaConexion.CerrarConexion();

nuevaConexion.AbrirConexion();

// Cadena SQL para eliminar un nuevo registro en la tabla Operativo.MaterialXServicio.

string delete = "DELETE FROM Operativo.MaterialXServicio WHERE idServicio = @IdServicio AND idMaterial = @IdMaterial";

SqlCommand cmd = new SqlCommand(delete, nuevaConexion.AbrirConexion());

cmd.Parameters.AddWithValue("@IdServicio", idServicio);

cmd.Parameters.AddWithValue("@IdMaterial", idMaterial);

cmd.ExecuteNonQuery();

nuevaConexion.CerrarConexion();

// Notificar que se ha actualizado.

OnMaterialXServicioUpdated();

}

/*

Dispara el Evento para notificar el cambio del subtotal.

*/

protected virtual void OnMaterialXServicioUpdated()

{

    SubtotalChanged?.Invoke(this, EventArgs.Empty);

}

/*

btnInserta_Click

Evento de clic del botón de inserción.

```

Llama a InsertaDato, actualiza el DataGridView llamando a ConsultaDatos y limpia los campos de entrada.

*/

private void btnInserta_Click(object sender, EventArgs e)

{

InsertaDato();

ConsultaDatos();

cmb_Material.SelectedIndex = -1;

txtCantidad.Clear();

}

/*

btnModifica_Click

Evento de clic del botón de modificación.

Llama a ModificaDato, actualiza el DataGridView llamando a ConsultaDatos y limpia los campos de entrada.

*/

private void btnModifica_Click(object sender, EventArgs e)

{

ModificaDato();

ConsultaDatos();

cmb_Material.SelectedIndex = -1;

txtCantidad.Clear();

}

/*

btnElimina_Click

Evento de clic del botón de eliminación.

Llama a EliminaDato, actualiza el DataGridView llamando a ConsultaDatos y limpia los campos de entrada.

*/

private void btnElimina_Click(object sender, EventArgs e)

{

EliminaDato();

ConsultaDatos();

cmb_Material.SelectedIndex = -1;

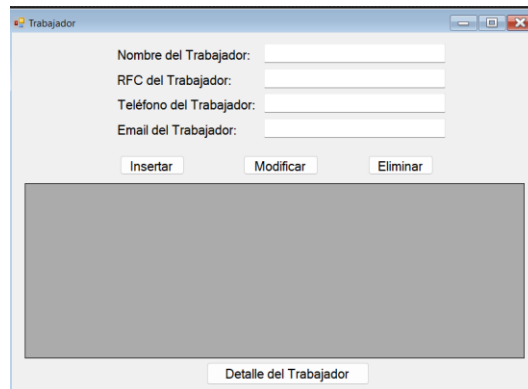
txtCantidad.Clear();

}

}

}

Formulario Trabajador.



Código documentado del Formulario Trabajador.

// Librerías del sistema y .NET Framework usadas para crear y gestionar el formulario Trabajador.

```
using System;
```

```
using System.Collections.Generic;
```

```
using System.ComponentModel;
```

```
using System.Data;
```

```
using System.Drawing;
```

```
using System.Linq;
```

```
using System.Text;
```

```
using System.Threading.Tasks;
```

```
using System.Windows.Forms;
```

```
using System.Data.SqlClient;
```

```
namespace ServiciosDeMantenimiento
```

```
{
```

```
    /*
```

```
        Clase para gestionar el manejo de los trabajadores dentro de los Servicios de Mantenimiento
```

```
        Proporciona métodos para consultar, insertar, modificar y eliminar datos.
```

```
    */
```

```
    public partial class Trabajador : Form
```

```
    {
```

```
        // Crea una instancia de una nueva conexión para conectarse con la base de datos de SQL Server.
```

```
        ConexionDB nuevaConexion = new ConexionDB("Server=ASUS\\SQLEXPRESS1; DATABASE = ServiciosdeMantenimiento; " +
```

```
            "integrated security = true");
```

```
        int idTrabajador = -1;
```



```

string nombreTrabajador = "";
string RFCTrabajador = "";
string telefonoTrabajador = "";
string emailTrabajador = "";

//Lista que almacena objetos de tipo DetalleTrabajador.
private List<DetalleTrabajador> detalleTrabajadorForms = new List<DetalleTrabajador>();

/*
    Constructor del formulario Trabajador.
    Inicializa el formulario y carga los datos de la tabla Trabajador en el DataGridView.
    También suscribe el evento CellClick de la tabla.
*/
public Trabajador()
{
    InitializeComponent();
    ConsultaDatos();
    TablaTrabajador.CellClick += TablaTrabajador_CellClick;
}

/*
    ConsultaDatos
    Recupera y muestra todos los registros de la tabla Operativo.Trabajador en el DataGridView.
    Formatea las columnas y limpia las filas antes de agregar los datos.
*/
public void ConsultaDatos()
{
    nuevaConexion.AbrirConexion();
    TablaTrabajador.DataSource = null;

    // Cadena SQL para consultar la tabla Operativo.Trabajador.
    string selectInfo = "SELECT * FROM Operativo.Trabajador";
    SqlCommand cmd = new SqlCommand(selectInfo, nuevaConexion.AbrirConexion());
    SqlDataReader reader = cmd.ExecuteReader();

    // Agrega columnas en la TablaTrabajador si es que no contiene columnas.
    if (TablaTrabajador.Columns.Count == 0)

```

```

{
    TablaTrabajador.Columns.Add("idTrabajador", "ID");
    TablaTrabajador.Columns.Add("nombreTrabajador", "Nombre del trabajador");
    TablaTrabajador.Columns.Add("RFCTrabajador", "RFC del trabajador");
    TablaTrabajador.Columns.Add("telefonoTrabajador", "Teléfono del trabajador");
    TablaTrabajador.Columns.Add("emailTrabajador", "Email del trabajador");
}

TablaTrabajador.Rows.Clear();

int i = 0;

// Agrega a TablaTrabajador una nueva fila para almacenar con los valores correspondientes a las columnas.
while (reader.Read())
{
    TablaTrabajador.Rows.Add();
    TablaTrabajador.Rows[i].Cells[0].Value = reader["idTrabajador"].ToString();
    TablaTrabajador.Rows[i].Cells[1].Value = reader["nombreTrabajador"].ToString();
    TablaTrabajador.Rows[i].Cells[2].Value = reader["RFCTrabajador"].ToString();
    TablaTrabajador.Rows[i].Cells[3].Value = reader["telefonoTrabajador"].ToString();
    TablaTrabajador.Rows[i].Cells[4].Value = reader["emailTrabajador"].ToString();
    i++;
}

reader.Close();
nuevaConexion.CerrarConexion();
}

/*
    TablaTrabajador_CellClick

    Evento que se activa al hacer clic en una celda del DataGridView.

    Llena los campos de entrada con los datos de la fila seleccionada.
*/

private void TablaTrabajador_CellClick(object sender, DataGridViewCellEventArgs e)
{
    if (e.RowIndex >= 0)
    {
        DataGridViewRow row = TablaTrabajador.Rows[e.RowIndex];
        idTrabajador = int.Parse(row.Cells[0].Value.ToString());
    }
}

```

```

txtNombre_Trabajador.Text = row.Cells[1].Value.ToString();

txtRFC_Trabajador.Text = row.Cells[2].Value.ToString();

txtTelefono_Trabajador.Text = row.Cells[3].Value.ToString();

txtEmail_Trabajador.Text = row.Cells[4].Value.ToString();


// Hace visible al botón btnDetalleTrabajador.

btnDetalleTrabajador.Visible = true;

}

}

/*
InsertaDato

Inserta un nuevo registro en la tabla Operativo.Trabajador con los datos ingresados.

Si ocurre un error (como duplicación de RFC, teléfono o email), muestra un mensaje de error.

*/

public void InsertaDato()
{
    nombreTrabajador = txtNombre_Trabajador.Text;

    RFCTrabajador = txtRFC_Trabajador.Text;

    telefonoTrabajador = txtTelefono_Trabajador.Text;

    emailTrabajador = txtEmail_Trabajador.Text;


    nuevaConexion.AbrirConexion();


    //Verifica si el RFC, el teléfono o el correo electrónico ya existen.

    string verifica = "SELECT COUNT(*) FROM Operativo.Trabajador WHERE RFCTrabajador=@RFC OR telefonoTrabajador = @Telefono OR emailTrabajador = @Email";

    SqlCommand checkCmd = new SqlCommand(verifica, nuevaConexion.AbrirConexion());

    checkCmd.Parameters.AddWithValue("@RFC", RFCTrabajador);

    checkCmd.Parameters.AddWithValue("@Telefono", telefonoTrabajador);

    checkCmd.Parameters.AddWithValue("@Email", emailTrabajador);

    int count = (int)checkCmd.ExecuteScalar();


    if (count > 0)

    {

        MessageBox.Show("El RFC, el teléfono o el correo electrónico ya están en uso.");

    }

}

```

```

else

{
    // Cadena SQL para agregar un nuevo registro en la tabla Operativo.Trabajador.

    string InsertInfo = "INSERT INTO Operativo.Trabajador(nombreTrabajador, RFCTrabajador, telefonoTrabajador,
emailTrabajador) " +

    "VALUES ('" + nombreTrabajador + "', '" + RFCTrabajador + "', '" + telefonoTrabajador + "', '" + emailTrabajador + "')";

    SqlCommand cm = new SqlCommand(InsertInfo, nuevaConexion.AbrirConexion());

    cm.ExecuteNonQuery();

}

nuevaConexion.CerrarConexion();

}

/*
ModificaDato

Modifica el registro seleccionado en la tabla Operativo.Trabajador con los datos actualizados.

Si ocurre un error (como duplicación de RFC, teléfono o email), muestra un mensaje de error.
*/

public void ModificaDato()

{

    nombreTrabajador = txtNombre_Trabajador.Text;

    RFCTrabajador = txtRFC_Trabajador.Text;

    telefonoTrabajador = txtTelefono_Trabajador.Text;

    emailTrabajador = txtEmail_Trabajador.Text;


    nuevaConexion.AbrirConexion();


    //Verifica si el RFC, el teléfono o el correo electrónico ya existen.

    string verifica = "SELECT COUNT(*) FROM Operativo.Trabajador WHERE RFCTrabajador=@RFC OR telefonoTrabajador =
@Telefono OR emailTrabajador = @Email";

    SqlCommand checkCmd = new SqlCommand(verifica, nuevaConexion.AbrirConexion());

    checkCmd.Parameters.AddWithValue("@RFC", RFCTrabajador);

    checkCmd.Parameters.AddWithValue("@Telefono", telefonoTrabajador);

    checkCmd.Parameters.AddWithValue("@Email", emailTrabajador);

    int count = (int)checkCmd.ExecuteScalar();

    if (count > 0)

    {

```

```

        MessageBox.Show("El RFC, el teléfono o el correo electrónico ya están en uso.");
    }

    else
    {
        // Cadena SQL para modificar un nuevo registro en la tabla Operativo.Trabajador.

        string update = "UPDATE Operativo.Trabajador SET nombreTrabajador = @Nombre, RFCTrabajador = @RFC
,telefonoTrabajador = @Telefono, emailTrabajador = @Email " +

        "WHERE idTrabajador = @Id";

        SqlCommand cmd = new SqlCommand(update, nuevaConexion.AbrirConexion());

        cmd.Parameters.AddWithValue("@Nombre", nombreTrabajador);

        cmd.Parameters.AddWithValue("@RFC", RFCTrabajador);

        cmd.Parameters.AddWithValue("@Telefono", telefonoTrabajador);

        cmd.Parameters.AddWithValue("@Email", emailTrabajador);

        cmd.Parameters.AddWithValue("@Id", idTrabajador);

        cmd.ExecuteNonQuery();

        nuevaConexion.CerrarConexion();

    }

    nuevaConexion.CerrarConexion();

}

/*
    EliminaDato

    Elimina el registro seleccionado en la tabla Operativo.Trabajador con los datos actualizados.
*/

public void EliminaDato()
{
    if (idTrabajador != -1)
    {
        nuevaConexion.AbrirConexion();

        // Cadena SQL para eliminar un nuevo registro en la tabla Operativo.Trabajador.

        string delete = "DELETE FROM Operativo.Trabajador WHERE idTrabajador = @Id";

        SqlCommand cmd = new SqlCommand(delete, nuevaConexion.AbrirConexion());

        cmd.Parameters.AddWithValue("@Id", idTrabajador);

        cmd.ExecuteNonQuery();

        string checkEmptyQuery = "SELECT COUNT(*) FROM Operativo.Trabajador";
    }
}

```

```

SqlCommand checkCmd = new SqlCommand(checkEmptyQuery, nuevaConexion.AbrirConexion());

int rowCount = (int)checkCmd.ExecuteScalar();

// Reinicia el valor del idTrabajador cuando la cantidad de filas es igual a 0.

if (rowCount == 0)
{
    string resetIdentityQuery = "DBCC CHECKIDENT ('Operativo.Trabajador', RESEED, 0)";

    SqlCommand resetCmd = new SqlCommand(resetIdentityQuery, nuevaConexion.AbrirConexion());

    resetCmd.ExecuteNonQuery();

}

nuevaConexion.CerrarConexion();

}

}

/*

btnInserta_Click

Evento de clic del botón de inserción.

Llama a InsertaDato, actualiza el DataGridView llamando a ConsultaDatos y limpia los campos de entrada.

*/

private void btnInserta_Click(object sender, EventArgs e)
{
    InsertaDato();

    ConsultaDatos();

    txtNombre_Trabajador.Clear();

    txtRFC_Trabajador.Clear();

    txtTelefono_Trabajador.Clear();

    txtEmail_Trabajador.Clear();

    idTrabajador = -1;

}

/*

btnModifica_Click

Evento de clic del botón de modificación.

Llama a ModificaDato, actualiza el DataGridView llamando a ConsultaDatos y limpia los campos de entrada.

*/

private void btnModifica_Click(object sender, EventArgs e)
{

```

```

ModificaDato();

ConsultaDatos();

txtNombre_Trabajador.Clear();

txtRFC_Trabajador.Clear();

txtTelefono_Trabajador.Clear();

txtEmail_Trabajador.Clear();

idTrabajador = -1;
}

/*
    btnElimina_Click

    Evento de clic del botón de eliminación.

    Llama a EliminaDato, actualiza el DataGridView llamando a ConsultaDatos y limpia los campos de entrada.
*/
private void btnElimina_Click(object sender, EventArgs e)
{
    EliminaDato();

    ConsultaDatos();

    txtNombre_Trabajador.Clear();

    txtRFC_Trabajador.Clear();

    txtTelefono_Trabajador.Clear();

    txtEmail_Trabajador.Clear();

    idTrabajador = -1;
}

/*
    Evento que se dispara al hacer clic en el botón de Detalle del Trabajador.

    Abre el formulario DetalleTrabajador para gestionar los detalles asociados a un trabajador específico.
*/
private void btnDetalleTrabajador_Click(object sender, EventArgs e)
{
    if (idTrabajador != -1)
    {
        DetalleTrabajador detalleTrabajadorForm = new DetalleTrabajador(idTrabajador);

        detalleTrabajadorForms.Add(detalleTrabajadorForm);

        detalleTrabajadorForm.Show();

        txtNombre_Trabajador.Clear();
    }
}

```

```

        txtRFC_Trabajador.Clear();

        txtTelefono_Trabajador.Clear();

        txtEmail_Trabajador.Clear();

        idTrabajador = -1;

    }

}

}

}

```

Formulario DetalleTrabajador.

Código documentado del Formulario DetalleTrabajador.

```

// Librerías del sistema y .NET Framework usadas para crear y gestionar el formulario DetalleTrabajador
using Microsoft.Win32;

using System;

using System.Data;

using System.Data.SqlClient;

using System.Security.Cryptography;

using System.Windows.Forms;

using static System.Windows.Forms.VisualStyles.VisualStyleElement.ListView;

namespace ServiciosDeMantenimiento
{
    /*
        Clase para gestionar el manejo de los detalles de los trabajadores dentro de los Servicios de Mantenimiento
        Proporciona métodos para consultar, insertar, modificar y eliminar datos.
    */

    public partial class DetalleTrabajador : Form
    {

```



```

// Crea una instancia de una nueva conexión para conectarse con la base de datos de SQL Server.

ConexionDB nuevaConexion = new ConexionDB("Server=ASUS\\SQLEXPRESS1; DATABASE = ServiciosdeMantenimiento;
integrated security = true");

int idTrabajador = -1;

string profesion = "";

string costoBase = "";

/*

Constructor del formulario DetalleTrabajador.

Inicializa el formulario y carga los datos de la tabla DetalleTrabajador en el DataGridView.

También suscribe el evento CellClick de la tabla.

*/

public DetalleTrabajador(int idTrabajador)
{
    InitializeComponent();

    this.idTrabajador = idTrabajador;

    ConsultarDatos();

    TablaDetalleTrabajador.CellClick += TablaDetalleTrabajador_CellClick;
}

/*

ConsultarDatos

Recupera y muestra todos los registros de la tabla Operativo.DetalleTrabajador en el DataGridView.

Formatea las columnas y limpia las filas antes de agregar los datos.

*/

public void ConsultarDatos()
{
    nuevaConexion.AbrirConexion();

    TablaDetalleTrabajador.DataSource = null;

    // Cadena SQL para consultar la tabla Operativo.DetalleTrabajador.

    string selectInfo = "SELECT dt.idTrabajador, dt.profesion, dt.costoBase, t.nombreTrabajador " +

        "FROM Operativo.DetalleTrabajador dt " +

        "JOIN Operativo.Trabajador t ON dt.idTrabajador = t.idTrabajador " +

        "WHERE dt.idTrabajador = @IdTrabajador";

    SqlCommand cmd = new SqlCommand(selectInfo, nuevaConexion.AbrirConexion());

    cmd.Parameters.AddWithValue("@IdTrabajador", idTrabajador);

```

```

SqlDataReader reader = cmd.ExecuteReader();

// Agrega columnas en la TablaDetalleTrabajador si es que no contiene columnas.
if (TablaDetalleTrabajador.Columns.Count == 0)
{
    TablaDetalleTrabajador.Columns.Add("idTrabajador", "Trabajador");
    TablaDetalleTrabajador.Columns.Add("profesion", "Profesión");
    TablaDetalleTrabajador.Columns.Add("costoBase", "Costo Base");

    TablaDetalleTrabajador.Columns["costoBase"].DefaultCellStyle.Format = "N2";
}

TablaDetalleTrabajador.Rows.Clear();
int i = 0;

// Agrega a TablaDetalleTrabajador una nueva fila para almacenar con los valores correspondientes a las columnas.
while (reader.Read())
{
    string nombreTrabajador = reader["nombreTrabajador"].ToString();
    string trabajador = nombreTrabajador;

    TablaDetalleTrabajador.Rows.Add();
    TablaDetalleTrabajador.Rows[i].Cells[0].Value = trabajador;
    TablaDetalleTrabajador.Rows[i].Cells[1].Value = reader["profesion"].ToString();
    TablaDetalleTrabajador.Rows[i].Cells[2].Value = Convert.ToDecimal(reader["costoBase"]).ToString("N2");
    i++;
}

reader.Close();
nuevaConexion.CerrarConexion();
}

/*
TablaDetalleTrabajador_CellClick

Evento que se activa al hacer clic en una celda del DataGridView.

Llena los campos de entrada con los datos de la fila seleccionada.
*/
private void TablaDetalleTrabajador_CellClick(object sender, DataGridViewCellEventArgs e)

```

```

{
    if (e.RowIndex >= 0)
    {
        DataGridViewRow row = TablaDetalleTrabajador.Rows[e.RowIndex];

        txtProfesion_Trabajador.Text = row.Cells[1].Value.ToString();

        txtCostoBase_Trabajador.Text = row.Cells[2].Value.ToString();
    }
}

/*
InsertaDato

Inserta un nuevo registro en la tabla Operativo.DetalleTrabajador con los datos ingresados.

Si ocurre un error(como duplicación de la profesión), muestra un mensaje de error.
*/

private void InsertaDato()
{
    profesion = txtProfesion_Trabajador.Text;
    costoBase = txtCostoBase_Trabajador.Text;

    //Verifica si profesión ya existe.
    string verificaProfesion = "SELECT COUNT(*) FROM Operativo.DetalleTrabajador " +
        "WHERE idTrabajador = @IdTrabajador AND profesion = @Profesion";

    SqlCommand checkCmd = new SqlCommand(verificaProfesion, nuevaConexion.AbrirConexion());
    checkCmd.Parameters.AddWithValue("@IdTrabajador", idTrabajador);
    checkCmd.Parameters.AddWithValue("@Profesion", profesion);

    int count = (int)checkCmd.ExecuteScalar();

    if (count > 0)
    {
        MessageBox.Show("Este trabajador ya tiene asignada esta profesión.");
        return;
    }

    nuevaConexion.AbrirConexion();

```

```

string InsertInfo = "INSERT INTO Operativo.DetalleTrabajador(idTrabajador, profesion, costoBase) " +
    "VALUES (@IdTrabajador, @Profesion, @CostoBase)";

SqlCommand cm = new SqlCommand(InsertInfo, nuevaConexion.AbrirConexion());

cm.Parameters.AddWithValue("@IdTrabajador", idTrabajador);

cm.Parameters.AddWithValue("@Profesion", profesion);

cm.Parameters.AddWithValue("@CostoBase", costoBase);

cm.ExecuteNonQuery();

nuevaConexion.CerrarConexion();
}

/*
ModificaDato

Modifica el registro seleccionado en la tabla Operativo.DetalleTrabajador con los datos actualizados.

Si ocurre un error(como duplicación de la profesión), muestra un mensaje de error.

*/

private void ModificaDato()
{
    DataGridViewRow selectedRow = TablaDetalleTrabajador.Rows[TablaDetalleTrabajador.SelectedCells[0].RowIndex];

    string profesionActual = selectedRow.Cells["profesion"].Value.ToString();

    string costoBaseActual = selectedRow.Cells["costoBase"].Value.ToString();

    profesion = txtProfesion_Trabajador.Text;

    costoBase = txtCostoBase_Trabajador.Text;

    if (profesion != profesionActual)
    {
        //Verifica si profesión ya existe.

        string verificaProfesion = "SELECT COUNT(*) FROM Operativo.DetalleTrabajador " +
            "WHERE idTrabajador = @IdTrabajador AND profesion = @Profesion";

        SqlCommand checkCmd = new SqlCommand(verificaProfesion, nuevaConexion.AbrirConexion());

        checkCmd.Parameters.AddWithValue("@IdTrabajador", idTrabajador);

        checkCmd.Parameters.AddWithValue("@Profesion", profesion);

        int count = (int)checkCmd.ExecuteScalar();

        nuevaConexion.CerrarConexion();

        if (count > 0)

```

```

    {
        MessageBox.Show("El trabajador ya tiene asignada esta profesión.");
        return;
    }
}

nuevaConexion.AbrirConexion();

// Cadena SQL para modificar un nuevo registro en la tabla Operativo.DetalleTrabajador.
string updateInfo = "UPDATE Operativo.DetalleTrabajador " +
    "SET profesion = @Profesion, costoBase = @CostoBase " +
    "WHERE idTrabajador = @IdTrabajador AND profesion = @ProfesionActual";

SqlCommand updateCmd = new SqlCommand(updateInfo, nuevaConexion.AbrirConexion());
updateCmd.Parameters.AddWithValue("@Profesion", profesion);
updateCmd.Parameters.AddWithValue("@CostoBase", costoBase);
updateCmd.Parameters.AddWithValue("@IdTrabajador", idTrabajador);
updateCmd.Parameters.AddWithValue("@ProfesionActual", profesionActual);

updateCmd.ExecuteNonQuery();
nuevaConexion.CerrarConexion();
}

/*
EliminaDato

Elimina el registro seleccionado en la tabla Operativo.DetalleTrabajador con los datos actualizados.
*/

public void EliminaDato()
{
    nuevaConexion.AbrirConexion();

    // Cadena SQL para eliminar un nuevo registro en la tabla Operativo.Trabajador.
    string delete = "DELETE FROM Operativo.DetalleTrabajador WHERE idTrabajador = @IdTrabajador AND profesion = @Profesion";

    SqlCommand cmd = new SqlCommand(delete, nuevaConexion.AbrirConexion());
    cmd.Parameters.AddWithValue("@IdTrabajador", idTrabajador);
    cmd.Parameters.AddWithValue("@Profesion", profesion);
    cmd.ExecuteNonQuery();
}

```

```

        nuevaConexion.CerrarConexion();
    }

    /**
     * btnInserta_Click
     *
     * Evento de clic del botón de inserción.
     *
     * Llama a InsertaDato, actualiza el DataGridView llamando a ConsultarDatos y limpia los campos de entrada.
     */
    private void btnInserta_Click(object sender, EventArgs e)
    {
        InsertarDato();
        ConsultarDatos();
        txtProfesion_Trabajador.Clear();
        txtCostoBase_Trabajador.Clear();
    }

    /**
     * btnModifica_Click
     *
     * Evento de clic del botón de modificación.
     *
     * Llama a ModificarDato, actualiza el DataGridView llamando a ConsultarDatos y limpia los campos de entrada.
     */
    private void btnModifica_Click(object sender, EventArgs e)
    {
        ModificarDato();
        ConsultarDatos();
        txtProfesion_Trabajador.Clear();
        txtCostoBase_Trabajador.Clear();
    }

    /**
     * btnElimina_Click
     *
     * Evento de clic del botón de eliminación.
     *
     * Llama a EliminarDato, actualiza el DataGridView llamando a ConsultarDatos y limpia los campos de entrada.
     */
    private void btnElimina_Click(object sender, EventArgs e)
    {
        EliminarDato();
    }

```

```

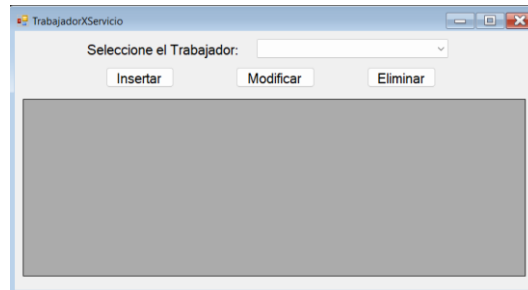
        ConsultarDatos();

        txtProfesion_Trabajador.Clear();

        txtCostoBase_Trabajador.Clear();
    }
}
}

```

Formulario TrabajadorXServicio.



Código documentado del Formulario TrabajadorXServicio.

// Librerías del sistema y .NET Framework usadas para crear y gestionar el formulario TrabajadorXServicio.

```

using System;

using System.Collections.Generic;

using System.ComponentModel;

using System.Data;

using System.Data.SqlClient;

using System.Drawing;

using System.Linq;

using System.Text;

using System.Threading.Tasks;

using System.Windows.Forms;

```

```

namespace ServiciosDeMantenimiento

```

```

{

```

```

    /*

```

```

        Clase para gestionar el manejo de los trabajadores de los servicios dentro de los Servicios de Mantenimiento

```

```

        Proporciona métodos para consultar, insertar, modificar y eliminar datos.

```

```

    */

```

```

    public partial class TrabajadorXServicio : Form

```

```

    {

```

```

        // Crea una instancia de una nueva conexión para conectarse con la base de datos de SQL Server.

```

```

ConexionDB nuevaConexion = new ConexionDB("Server=ASUS\\SQLEXPRESS1; DATABASE = ServiciosdeMantenimiento;" +
    "integrated security = true");

int idTrabajadorXServicio = -1;

int idServicio;

string idTrabajador = "";

float subtotalTrabajo = 0;

// Define un evento llamado SubtotalChanged el cual se utiliza para notificar cuando ocurre un cambio en el subtotal de un
servicio.

public event EventHandler SubtotalChanged;

/*
    Constructor del formulario TrabajadorXServicio.

    Inicializa el formulario y carga los datos de la tabla TrabajadorXServicio en el DataGridView.

    También suscribe el evento CellClick de la tabla.
*/
public TrabajadorXServicio(int idServicio)
{
    InitializeComponent();

    this.idServicio = idServicio;

    ConsultarDatos();

    CargarTrabajadores();

    TablaTrabajadorXServicio.CellClick += TablaTrabajadorXServicio_CellClick;
}

/*
    ConsultarDatos

    Recupera y muestra todos los registros de la tabla Operativo.TrabajadorXServicio en el DataGridView.

    Formatea la columna de precios y limpia las filas antes de agregar los datos.
*/
public void ConsultarDatos()
{
    nuevaConexion.AbrirConexion();

    TablaTrabajadorXServicio.DataSource = null;

    // Cadena SQL para consultar la tabla Operativo.TrabajadorXServicio.

    string selectInfo = @"SELECT txs.idTrabajadorXServicio, txs.idServicio, txs.idTrabajador, txs.subtotalTrabajo,

```



```

        CONCAT(t.idServicio, ' - ', c.nombreCliente) AS ServicioCliente,
        CONCAT(tr.nombreTrabajador, ' (', COUNT(dt.idTrabajador), ')') AS TrabajadorConProfesiones
    FROM Operativo.TrabajadorXServicio txs
    JOIN Operativo.Trabajador tr ON txs.idTrabajador = tr.idTrabajador
    JOIN Operativo.Servicio t ON txs.idServicio = t.idServicio
    JOIN Comercial.DomicilioCliente d ON t.idDomicilio = d.idDomicilio
    JOIN Comercial.Cliente c ON d.idCliente = c.idCliente
    LEFT JOIN Operativo.DetalleTrabajador dt ON tr.idTrabajador = dt.idTrabajador
    WHERE t.idServicio = @IdServicio
    GROUP BY txs.idTrabajadorXServicio, txs.idServicio, txs.idTrabajador, txs.subtotalTrabajo,
    t.idServicio, c.nombreCliente, tr.nombreTrabajador";

SqlCommand cmd = new SqlCommand(selectInfo, nuevaConexion.AbrirConexion());
cmd.Parameters.AddWithValue("@IdServicio", idServicio);
SqlDataReader reader = cmd.ExecuteReader();

// Agrega columnas en la TablaTrabajadorXServicio si es que no contiene columnas.
if (TablaTrabajadorXServicio.Columns.Count == 0)
{
    TablaTrabajadorXServicio.Columns.Add("idTrabajadorXServicio", "ID");
    TablaTrabajadorXServicio.Columns.Add("ServicioCliente", "Servicio");
    TablaTrabajadorXServicio.Columns.Add("TrabajadorConProfesiones", "Trabajador");
    TablaTrabajadorXServicio.Columns.Add("subtotalTrabajo", "Subtotal");

    TablaTrabajadorXServicio.Columns["subtotalTrabajo"].DefaultCellStyle.Format = "N2";
}

TablaTrabajadorXServicio.Rows.Clear();
int i = 0;

// Agrega a TablaTrabajadorXServicio una nueva fila para almacenar con los valores correspondientes a las columnas.
while (reader.Read())
{
    string servicioCliente = reader["ServicioCliente"].ToString();
    string trabajadorConProfesiones = reader["TrabajadorConProfesiones"].ToString();
    decimal subtotalTrabajo = reader["subtotalTrabajo"] != DBNull.Value ? Convert.ToDecimal(reader["subtotalTrabajo"]) : 0;

```

```

        TablaTrabajadorXServicio.Rows.Add();

        TablaTrabajadorXServicio.Rows[i].Cells[0].Value = reader["idTrabajadorXServicio"].ToString();

        TablaTrabajadorXServicio.Rows[i].Cells[1].Value = servicioCliente;

        TablaTrabajadorXServicio.Rows[i].Cells[2].Value = trabajadorConProfesiones;

        TablaTrabajadorXServicio.Rows[i].Cells[3].Value = subtotalTrabajo.ToString("N2");

        i++;

        TablaTrabajadorXServicio.Columns["idTrabajadorXServicio"].Visible = false;
    }

    reader.Close();

    nuevaConexion.CerrarConexion();
}

/*
    Carga la información de la tabla Operativo.Trabajador y la muestra en un combobox.
*/

public void CargarTrabajadores()
{
    nuevaConexion.AbrirConexion();

    string selectInfo = @"SELECT t.NombreTrabajador, COUNT(dt.profesion) AS CantidadProfesiones
        FROM Operativo.Trabajador t
        LEFT JOIN Operativo.DetalleTrabajador dt ON t.idTrabajador = dt.idTrabajador
        GROUP BY t.NombreTrabajador";

    SqlCommand cmd = new SqlCommand(selectInfo, nuevaConexion.AbrirConexion());

    SqlDataReader reader = cmd.ExecuteReader();

    cmb_Trabajador.Items.Clear();

    // Lee cada trabajador del resultado de la consulta
    while (reader.Read())
    {
        string nombreTrabajador = reader["NombreTrabajador"].ToString();

        int cantidadProfesiones = Convert.ToInt32(reader["CantidadProfesiones"]);

        string trabajador = $"{nombreTrabajador} {{cantidadProfesiones}}";
    }
}

```

```

        // Agrega el resultado al combo box
        cmb_Trabajador.Items.Add(trabajador);
    }

    reader.Close();
    nuevaConexion.CerrarConexion();
}

/*
    TablaTrabajadorXServicio_CellClick

    Evento que se activa al hacer clic en una celda del DataGridView.

    Llena los campos de entrada con los datos de la fila seleccionada.
*/
private void TablaTrabajadorXServicio_CellClick(object sender, DataGridViewCellEventArgs e)
{
    if (e.RowIndex >= 0)
    {
        DataGridViewRow row = TablaTrabajadorXServicio.Rows[e.RowIndex];

        idTrabajadorXServicio = int.Parse(row.Cells[0].Value.ToString());

        cmb_Trabajador.Text = row.Cells[2].Value.ToString();
    }
}

/*
    Recupera el valor de la columna del idTrabajador.
*/
private int ObtenerIdTrabajador(string nombreTrabajador)
{
    nuevaConexion.AbrirConexion();

    string queryIdTrabajador = "SELECT idTrabajador FROM Operativo.Trabajador WHERE NombreTrabajador = @Nombre";

    SqlCommand cmdIdTrabajador = new SqlCommand(queryIdTrabajador, nuevaConexion.AbrirConexion());

    cmdIdTrabajador.Parameters.AddWithValue("@Nombre", nombreTrabajador);

    var resultado = cmdIdTrabajador.ExecuteScalar();

    return resultado != null ? Convert.ToInt32(resultado) : -1;
}

```

```

/*
    InsertaDato

    Inserta un nuevo registro en la tabla Operativo.TrabajadorXServicio con los datos ingresados.
*/
public void InsertaDato()
{
    string trabajadorSeleccionado = cmb_Trabajador.SelectedItem.ToString();

    string nombreTrabajador = trabajadorSeleccionado.Split(' ')[0].Trim();

    nuevaConexion.AbrirConexion();

    //Rescatar la concatenación del trabajador.
    string queryIdTrabajador = "SELECT idTrabajador FROM Operativo.Trabajador WHERE NombreTrabajador = @NombreTrabajador";

    SqlCommand cmdIdTrabajador = new SqlCommand(queryIdTrabajador, nuevaConexion.AbrirConexion());

    cmdIdTrabajador.Parameters.AddWithValue("@NombreTrabajador", nombreTrabajador);

    var resultado = cmdIdTrabajador.ExecuteScalar();

    nuevaConexion.CerrarConexion();

    if (resultado == null)
    {
        MessageBox.Show("Trabajador no encontrado.");

        return;
    }

    string idTrabajador = resultado.ToString();

    nuevaConexion.AbrirConexion();

    // Verificar si ya existe un trabajador para este servicio.
    string queryProfesiones = "SELECT COUNT(*) FROM Operativo.DetalleTrabajador WHERE idTrabajador = @IdTrabajador";

    SqlCommand cmdProfesiones = new SqlCommand(queryProfesiones, nuevaConexion.AbrirConexion());

    cmdProfesiones.Parameters.AddWithValue("@IdTrabajador", idTrabajador);

    int cantidadProfesiones = (int)cmdProfesiones.ExecuteScalar();

    nuevaConexion.CerrarConexion();

```

```

string trabajadorConProfesiones = $"{nombreTrabajador} {{cantidadProfesiones} profesiones}";

nuevaConexion.AbrirConexion();

// Verificar si ya existe un trabajadorxservicio.

string verificarExistencia = "SELECT COUNT(*) FROM Operativo.TrabajadorXServicio WHERE idServicio = @IdServicio AND
idTrabajador = @IdTrabajador";

SqlCommand cmdVerificar = new SqlCommand(verificarExistencia, nuevaConexion.AbrirConexion());

cmdVerificar.Parameters.AddWithValue("@IdServicio", idServicio);

cmdVerificar.Parameters.AddWithValue("@IdTrabajador", idTrabajador);

int count = (int)cmdVerificar.ExecuteScalar();

nuevaConexion.CerrarConexion();

if (count > 0)
{
    MessageBox.Show("El trabajador ya está asociado a este servicio.");

    return;
}

nuevaConexion.AbrirConexion();

// Cadena SQL para agregar un nuevo registro en la tabla Operativo.TrabajadorXServicio.

string insertInfo = "INSERT INTO Operativo.TrabajadorXServicio(idServicio, idTrabajador, subtotalTrabajo) VALUES
(@IdServicio, @IdTrabajador, @Subtotal)";

SqlCommand cm = new SqlCommand(insertInfo, nuevaConexion.AbrirConexion());

cm.Parameters.AddWithValue("@IdServicio", idServicio);

cm.Parameters.AddWithValue("@IdTrabajador", idTrabajador);

cm.Parameters.AddWithValue("@Subtotal", subtotalTrabajo);

cm.ExecuteNonQuery();

nuevaConexion.CerrarConexion();

// Notificar que se ha actualizado.

OnTrabajadorXServicioUpdated();
}

```

```

/*
ModificaDato

Modifica el registro seleccionado en la tabla Operativo.TrabajadorXServicio con los datos actualizados.
*/

public void ModificaDato()
{
    idTrabajador = cmb_Trabajador.Text;

    string nombreTrabajador = idTrabajador.Split(',')[0].Trim();

    nuevaConexion.AbrirConexion();

    //Rescatar la concatenación del trabajador.

    string queryIdTrabajador = "SELECT idTrabajador FROM Operativo.Trabajador WHERE NombreTrabajador = @Nombre";
    SqlCommand cmdIdTrabajador = new SqlCommand(queryIdTrabajador, nuevaConexion.AbrirConexion());
    cmdIdTrabajador.Parameters.AddWithValue("@Nombre", nombreTrabajador);

    var resultado = cmdIdTrabajador.ExecuteScalar();
    nuevaConexion.CerrarConexion();

    if (resultado == null)
    {
        MessageBox.Show("Trabajador no encontrado.");
        return;
    }

    idTrabajador = resultado.ToString();

    nuevaConexion.AbrirConexion();

    // Verificar si ya existe un trabajador para este servicio.

    string queryOldIdTrabajador = "SELECT idTrabajador FROM Operativo.TrabajadorXServicio WHERE idServicio = @IdServicio";
    SqlCommand cmdOldIdTrabajador = new SqlCommand(queryOldIdTrabajador, nuevaConexion.AbrirConexion());
    cmdOldIdTrabajador.Parameters.AddWithValue("@IdServicio", idServicio);

    var resultadoOldId = cmdOldIdTrabajador.ExecuteScalar();
    nuevaConexion.CerrarConexion();

    if (resultadoOldId == null)

```

```

{
    MessageBox.Show("No se encontró una asociación con este servicio.");
    return;
}

string oldIdTrabajador = resultadoOldId.ToString();

nuevaConexion.AbrirConexion();

// Verificar si ya existe un trabajadorxservicio.

string verificarExistencia = "SELECT COUNT(*) FROM Operativo.TrabajadorXServicio WHERE idServicio = @IdServicio AND
idTrabajador = @IdTrabajador";

SqlCommand cmdVerificar = new SqlCommand(verificarExistencia, nuevaConexion.AbrirConexion());
cmdVerificar.Parameters.AddWithValue("@IdServicio", idServicio);
cmdVerificar.Parameters.AddWithValue("@IdTrabajador", idTrabajador);

int count = (int)cmdVerificar.ExecuteScalar();
nuevaConexion.CerrarConexion();

if (count > 0)
{
    MessageBox.Show("El trabajador ya está asociado a este servicio.");
    return;
}

nuevaConexion.AbrirConexion();

// Cadena SQL para modificar un nuevo registro en la tabla Operativo.TrabajadorXServicio.

string updateQuery = "UPDATE Operativo.TrabajadorXServicio SET idTrabajador = @Trabajador WHERE idTrabajadorXServicio
= @Id";

SqlCommand cmdUpdate = new SqlCommand(updateQuery, nuevaConexion.AbrirConexion());
cmdUpdate.Parameters.AddWithValue("@Trabajador", idTrabajador);
cmdUpdate.Parameters.AddWithValue("@IdServicio", idServicio);
cmdUpdate.Parameters.AddWithValue("@Id", idTrabajadorXServicio);
int rowsAffected = cmdUpdate.ExecuteNonQuery();
nuevaConexion.CerrarConexion();

```

```

        // Notificar que se ha actualizado.
        OnTrabajadorXServicioUpdated();
    }

    /*
    EliminaDato
    Elimina el registro seleccionado en la tabla Operativo.TrabajadorXServicio con los datos actualizados.
    */
    public void EliminaDato()
    {
        string trabajadorSeleccionado = cmb_Trabajador.Text;
        string nombreTrabajador = trabajadorSeleccionado.Split('(')[0].Trim();

        nuevaConexion.AbrirConexion();

        //Rescatar la concatenación del trabajador.
        string queryIdTrabajador = "SELECT idTrabajador FROM Operativo.Trabajador WHERE NombreTrabajador = @Nombre";
        SqlCommand cmdIdTrabajador = new SqlCommand(queryIdTrabajador, nuevaConexion.AbrirConexion());
        cmdIdTrabajador.Parameters.AddWithValue("@Nombre", nombreTrabajador);

        var resultado = cmdIdTrabajador.ExecuteScalar();
        nuevaConexion.CerrarConexion();

        if (resultado == null)
        {
            MessageBox.Show("Trabajador no encontrado.");
            return;
        }

        idTrabajador = resultado.ToString();

        nuevaConexion.AbrirConexion();

        // Cadena SQL para eliminar un nuevo registro en la tabla Operativo.TrabajadorXServicio.
        string delete = "DELETE FROM Operativo.TrabajadorXServicio WHERE idServicio = @IdServicio AND idTrabajador = @IdTrabajador";

        SqlCommand cmdDelete = new SqlCommand(delete, nuevaConexion.AbrirConexion());
    
```



```

cmdDelete.Parameters.AddWithValue("@IdServicio", idServicio);

cmdDelete.Parameters.AddWithValue("@IdTrabajador", idTrabajador);

int rowsAffected = cmdDelete.ExecuteNonQuery();

nuevaConexion.CerrarConexion();

// Notificar que se ha actualizado.
OnTrabajadorXServicioUpdated();
}

/*
Dispara el Evento para notificar el cambio del subtotal.
*/

protected virtual void OnTrabajadorXServicioUpdated()
{
    SubtotalChanged?.Invoke(this, EventArgs.Empty);
}

/*
btnInserta_Click

Evento de clic del botón de inserción.

Llama a InsertaDato, actualiza el DataGridView llamando a ConsultaDatos y limpia los campos de entrada.
*/

private void btnInserta_Click(object sender, EventArgs e)
{
    InsertaDato();
    ConsultaDatos();
    cmb_Trabajador.SelectedIndex = -1;
}

/*
btnModifica_Click

Evento de clic del botón de modificación.

Llama a ModificaDato, actualiza el DataGridView llamando a ConsultaDatos y limpia los campos de entrada.
*/

private void btnModifica_Click(object sender, EventArgs e)
{
    ModificaDato();
}

```

```
        ConsultarDatos();  
        cmb_Trabajador.SelectedIndex = -1;  
    }  
  
    /*  
    btnEliminar_Click  
    Evento de clic del botón de eliminación.  
    Llama a EliminarDato, actualiza el DataGridView llamando a ConsultarDatos y limpia los campos de entrada.  
    */  
    private void btnEliminar_Click(object sender, EventArgs e)  
    {  
        EliminarDato();  
        ConsultarDatos();  
        cmb_Trabajador.SelectedIndex = -1;  
    }  
}  
}
```